

Εργασία NLP 3: Named-Entity Recognition using BERT

Μάνθος Γεώργιος, MTN2513

To notebook είναι port του NER-BERT.py .

Το παρακάτω κελί περιέχει την αρχικοποίηση των βιβλιοθηκών, τον ορισμό των υπερπαραμέτρων (hyper-parameters), τη φόρτωση του dataset και τις συναρτήσεις προεπεξεργασίας (όπως η `encode` και η `EvaluateModel`), αντλημένα από το αρχικό script.

Προετοιμασία

```
In [ ]: %pip install ipykernel
%pip install torch
%pip install transformers
%pip install scikit-learn
%pip install sequeval
%pip install tqdm
%pip install numpy
%pip install openai

In [ ]: # dependencies
import torch
import torch.optim as optim
from transformers import BertForTokenClassification, BertTokenizerFast
from sklearn.metrics import accuracy_score, balanced_accuracy_score, classification_report
from sequeval.metrics import classification_report as sequeval_report
from sequeval.metrics import f1_score as sequeval_f1
from sequeval.metrics import precision_score as sequeval_precision
from sequeval.metrics import recall_score as sequeval_recall
from tqdm.auto import tqdm
import numpy as np
import time

# hyper-parameters
EPOCHS = 3
BATCH_SIZE = 8
LR = 1e-5
NUM_ITERATIONS = 3 # Για το Task 1

# the path of the data files
base_path = './conll003-englishversion/'

# use GPU if available
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# read the data files
def load_sentences(filepath):
    sentences = []
    tokens, pos_tags, chunk_tags, ner_tags = [], [], [], []
    with open(filepath, 'r', encoding='utf-8') as f:
        for line in f.readlines():
            if (line.startswith('-DOCSTART-') or line.strip() == ''):
                if len(tokens) > 0:
                    sentences.append({'tokens': tokens, 'pos_tags': pos_tags, 'chunk_tags': chunk_tags, 'ner_tags':
tokens, pos_tags, chunk_tags, ner_tags = [], [], [], []
                else:
                    l = line.strip().split(' ')
                    if len(l) >= 4:
                        tokens.append(l[0])
                        pos_tags.append(l[1])
                        chunk_tags.append(l[2])
                        ner_tags.append(l[3])
            if len(tokens) > 0:
                sentences.append({'tokens': tokens, 'pos_tags': pos_tags, 'chunk_tags': chunk_tags, 'ner_tags': ner_tags})
    return sentences

print('loading data')
train_sentences = load_sentences(base_path + 'train.txt')
test_sentences = load_sentences(base_path + 'test.txt')
valid_sentences = load_sentences(base_path + 'valid.txt')

# build tag set and label mappings
all_tags = sorted({tag for s in train_sentences for tag in s['ner_tags']})
label2id = {tag: i for i, tag in enumerate(all_tags)}
id2label = {i: tag for tag, i in label2id.items()}
num_labels = len(all_tags)
print('Tagset size:', num_labels)
print('Tags:', all_tags)

# load BERT tokenizer
```

```

bert_version = 'bert-base-uncased'
tokenizer = BertTokenizerFast.from_pretrained(bert_version)

def align_label(tokens, labels):
    word_ids = tokens.word_ids()
    previous_word_idx = None
    label_ids = []
    for word_idx in word_ids:
        if word_idx is None:
            label_ids.append(-100)
        elif word_idx != previous_word_idx:
            label_ids.append(label2id.get(labels[word_idx], -100))
        else:
            label_ids.append(-100)
        previous_word_idx = word_idx
    return label_ids

def encode(sentence):
    encodings = tokenizer(
        sentence['tokens'], truncation=True, padding='max_length',
        is_split_into_words=True, return_tensors='pt'
    )
    labels = align_label(encodings, sentence['ner_tags'])
    return {
        'input_ids': encodings['input_ids'].squeeze(0),
        'attention_mask': encodings['attention_mask'].squeeze(0),
        'labels': torch.tensor(labels, dtype=torch.long)
    }

print('encoding data')
train_dataset = [encode(sentence) for sentence in train_sentences]
valid_dataset = [encode(sentence) for sentence in valid_sentences]
test_dataset = [encode(sentence) for sentence in test_sentences]

class InputDataset(torch.utils.data.Dataset):
    def __init__(self, data): self.data = data
    def __len__(self): return len(self.data)
    def __getitem__(self, idx): return self.data[idx]

train_dataset = InputDataset(train_dataset)
valid_dataset = InputDataset(valid_dataset)
test_dataset = InputDataset(test_dataset)

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
valid_loader = torch.utils.data.DataLoader(valid_dataset, batch_size=BATCH_SIZE)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=BATCH_SIZE)

def EvaluateModel(model, data_loader):
    model.eval()
    Y_actual_flat, Y_preds_flat = [], []
    y_true_tags, y_pred_tags = [], []

    with torch.no_grad():
        for batch in data_loader:
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            logits = outputs.logits
            preds = torch.argmax(logits, dim=-1)

            for idx in range(batch['labels'].size(0)):
                true_values_all = batch['labels'][idx]
                mask = (true_values_all != -100)
                true_values = true_values_all[mask]
                pred_values = preds[idx][mask]

                Y_actual_flat.append(true_values)
                Y_preds_flat.append(pred_values)

            y_true_tags.append([id2label[i] for i in true_values.tolist()])
            y_pred_tags.append([id2label[i] for i in pred_values.tolist()])

    Y_actual_flat = torch.cat(Y_actual_flat).detach().cpu().numpy()
    Y_preds_flat = torch.cat(Y_preds_flat).detach().cpu().numpy()

    return Y_actual_flat, Y_preds_flat, y_true_tags, y_pred_tags

```

Task 1: Training for Multiple Iterations

Ανάλυση Κώδικα

Στο παρακάτω κελί κώδικα προσάρμοσα τον αρχικό βρόχο εκπαίδευσης (training loop) ώστε να εκτελείται 3 ανεξάρτητες φορές (iterations). Συγκεκριμένα:

1. Προστέθηκε ένας εξωτερικός βρόγχος `for iteration in range(NUM_ITERATIONS):`.
2. Μέσα σε αυτόν τον βρόγχο, **αρχικοποιείται εξ αρχής** το μοντέλο (`BertForTokenClassification`) και ο AdamW optimizer σε

κάθε επανάληψη, έτσι ώστε να εξασφαλίζεται ότι το μοντέλο ξεκινάει "καθαρό" από την αρχή και δεν συνεχίζει την εκπαίδευση από το προηγούμενο iteration.

- Χρησιμοποιείται η συνάρτηση `time.time()` πριν και μετά τον βρόχο των εποχών (epochs) για να υπολογίσουμε το χρόνο εκπαίδευσης κάθε επανάληψης.
- Στη συνέχεια, τρέχει αξιολόγηση στο test set χρησιμοποιώντας την `EvaluateModel`. Υπολογίζονται:
 - Token-level micro accuracy:** μέσω της `accuracy_score` της βιβλιοθήκης `sklearn`.
 - Token-level macro accuracy:** μέσω της `balanced_accuracy_score` της `sklearn`.
 - Entity-level micro/macro F1:** μέσω της `sequeval.metrics.f1_score`, περνώντας την παράμετρο `average='micro'` και `average='macro'` αντίστοιχα.
- Στο τέλος, χρησιμοποιώντας τη βιβλιοθήκη `numpy` (`np.mean` και `np.std`), υπολογίζεται και τυπώνεται η μέση τιμή (mean) και η τυπική απόκλιση (standard deviation) για όλες τις παραπάνω μετρικές.

```
In [2]: # =====
# TASK 1: Multiple Iterations & Evaluation
# =====

all_token_micro_acc = []
all_token_macro_acc = []
all_entity_micro_f1 = []
all_entity_macro_f1 = []
all_training_times = []

print('Training the model for multiple iterations...')

for iteration in range(NUM_ITERATIONS):
    print(f'\n=====')
    print(f'          ITERATION {iteration + 1}/{NUM_ITERATIONS}')
    print(f'=====')

    # Επαναρχικοποίηση μοντέλου και optimizer από την αρχή
    model = BertForTokenClassification.from_pretrained(
        bert_version,
        num_labels=num_labels,
        id2label=id2label,
        label2id=label2id
    )
    model.to(device)
    optimizer = optim.AdamW(params=model.parameters(), lr=LR)

    start_time = time.time()

    for epoch in range(EPOCHS):
        model.train()
        print(f'Epoch {epoch + 1}/{EPOCHS}')
        for batch in tqdm(train_loader, desc=f"Iter {iteration+1} - Epoch {epoch + 1}"):
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            loss = outputs.loss

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

    end_time = time.time()
    training_time = end_time - start_time
    all_training_times.append(training_time)
    print(f"Training Time for Iteration {iteration + 1}: {training_time:.2f} seconds")

    print('Applying the model to the test set...')
    Y_actual, Y_preds, y_true_tags, y_pred_tags = EvaluateModel(model, test_loader)

    # Υπολογισμός Token-level μετρικών
    tok_micro = accuracy_score(Y_actual, Y_preds)
    tok_macro = balanced_accuracy_score(Y_actual, Y_preds)

    # Υπολογισμός Entity-level μετρικών
    ent_micro = sequeval_f1(y_true_tags, y_pred_tags, average='micro')
    ent_macro = sequeval_f1(y_true_tags, y_pred_tags, average='macro')

    all_token_micro_acc.append(tok_micro)
    all_token_macro_acc.append(tok_macro)
    all_entity_micro_f1.append(ent_micro)
    all_entity_macro_f1.append(ent_macro)

    print(f"--> Token Micro Acc : {tok_micro:.4f}")
    print(f"--> Token Macro Acc : {tok_macro:.4f}")
    print(f"--> Entity Micro F1 : {ent_micro:.4f}")
    print(f"--> Entity Macro F1 : {ent_macro:.4f}")

# Υπολογισμός και εκτύπωση Mean & Std
print('\n=== FINAL STATISTICS OVER 3 ITERATIONS ===')
print(f"Training Time          : Mean = {np.mean(all_training_times):.2f} s, Std = {np.std(all_training_times):.2f} s")
print(f"Token-level Micro Acc   : Mean = {np.mean(all_token_micro_acc):.4f}, Std = {np.std(all_token_micro_acc):.4f}")
```

```
print(f"Token-level Macro Acc : Mean = {np.mean(all_token_macro_acc):.4f}, Std = {np.std(all_token_macro_acc):.4f}")
print(f"Entity-level Micro F1 : Mean = {np.mean(all_entity_micro_f1):.4f}, Std = {np.std(all_entity_micro_f1):.4f}")
print(f"Entity-level Macro F1 : Mean = {np.mean(all_entity_macro_f1):.4f}, Std = {np.std(all_entity_macro_f1):.4f}")
```

Training the model for multiple iterations...

=====
 ITERATION 1/3
=====

```
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 1199.68it/s]
[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
bert.pooler.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
bert.pooler.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your d
ownstream task.
Epoch 1/3
Iter 1 - Epoch 1: 100%|██████████| 1756/1756 [12:19<00:00, 2.38it/s]
Epoch 2/3
Iter 1 - Epoch 2: 100%|██████████| 1756/1756 [12:17<00:00, 2.38it/s]
Epoch 3/3
Iter 1 - Epoch 3: 100%|██████████| 1756/1756 [12:17<00:00, 2.38it/s]
Training Time for Iteration 1: 2215.09 seconds
Applying the model to the test set...
--> Token Micro Acc : 0.9801
--> Token Macro Acc : 0.9079
--> Entity Micro F1 : 0.9014
--> Entity Macro F1 : 0.8846
```

=====
 ITERATION 2/3
=====

```
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 10925.56it/s]
[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
bert.pooler.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
bert.pooler.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your d
ownstream task.
Epoch 1/3
Iter 2 - Epoch 1: 100%|██████████| 1756/1756 [12:17<00:00, 2.38it/s]
Epoch 2/3
Iter 2 - Epoch 2: 100%|██████████| 1756/1756 [12:18<00:00, 2.38it/s]
Epoch 3/3
Iter 2 - Epoch 3: 100%|██████████| 1756/1756 [12:02<00:00, 2.43it/s]
Training Time for Iteration 2: 2199.21 seconds
Applying the model to the test set...
--> Token Micro Acc : 0.9795
--> Token Macro Acc : 0.9085
--> Entity Micro F1 : 0.8996
--> Entity Macro F1 : 0.8819
```

=====
 ITERATION 3/3
=====

```
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 11250.91it/s]
[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
bert.pooler.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
bert.pooler.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your d
ownstream task.
Epoch 1/3
Iter 3 - Epoch 1: 100%|██████████| 1756/1756 [12:02<00:00, 2.43it/s]
Epoch 2/3
Iter 3 - Epoch 2: 100%|██████████| 1756/1756 [12:01<00:00, 2.43it/s]
Epoch 3/3
Iter 3 - Epoch 3: 100%|██████████| 1756/1756 [12:01<00:00, 2.43it/s]
Training Time for Iteration 3: 2165.52 seconds
Applying the model to the test set...
--> Token Micro Acc : 0.9795
--> Token Macro Acc : 0.8936
--> Entity Micro F1 : 0.8914
--> Entity Macro F1 : 0.8751

=== FINAL STATISTICS OVER 3 ITERATIONS ===
Training Time : Mean = 2193.27 s, Std = 20.67 s
Token-level Micro Acc : Mean = 0.9797, Std = 0.0003
Token-level Macro Acc : Mean = 0.9033, Std = 0.0069
Entity-level Micro F1 : Mean = 0.8974, Std = 0.0044
Entity-level Macro F1 : Mean = 0.8805, Std = 0.0040
```

Τα τελικά στατιστικά διαμορφώνονται ως εξής:

Metric	Mean	Std Dev
Training Time	2193.27 s	20.67 s
Token-level Micro Acc	0.9797	0.0003
Token-level Macro Acc	0.9033	0.0069
Entity-level Micro F1	0.8974	0.0044
Entity-level Macro F1	0.8805	0.0040

Το training time των ~12 λεπτών ανά εποχή επιτεύχθηκε χρησιμοποιώντας μία rtx 4060, η οποία αντικατέστησε την rx480 που χρησιμοποιήθηκε για την εκπόνηση της προηγούμενης εργασίας.

Ερώτημα 2

Διαφορές μεταξύ token-level και entity level μετρικών αξιολόγησης\

Η **Token-level αξιολόγηση** εξετάζει το κάθε token εντελώς ανεξάρτητα από τα υπόλοιπα, δηλαδή ελέγχει αν η μεμονωμένη λέξη ταξινομήθηκε σωστά (π.χ. αν ένα token προβλέφθηκε ως 'B-PER' αντί για 'O'). Αν και είναι χρήσιμη κατά τον υπολογισμό του loss, συχνά δίνει παραπλανητικά υψηλά ποσοστά (π.χ. στην ακρίβεια) επειδή η συντριπτική πλειοψηφία των tokens σε ένα κείμενο ανήκει στην κατηγορία 'O' (Outside), δημιουργώντας μεγάλη ανισορροπία κλάσεων. Επιπλέον σε περιπτώσεις όπως ονόματα, αν ένα μοντέλο βρει σωστά το μισό όνομα (π.χ. το μικρό αλλά όχι το επώνυμο), στο token-level evaluation θα θεωρηθεί εν μέρει σωστό.

Από την άλλη, η **Entity-level αξιολόγηση** (τουλάχιστον όπως παρέχεται από τη βιβλιοθήκη `seqeval`) εξετάζει την κάθε οντότητα ως ένα αδιαίρετο σύνολο. Για να θεωρηθεί "σωστή" μια πρόβλεψη σε entity-level, πρέπει το μοντέλο να προβλέψει ταυτόχρονα και τα ακριβή όρια της οντότητας (από ποια λέξη ξεκινάει και σε ποια τελειώνει), και τον σωστό τύπο (π.χ. PER, LOC, ORG). Αν ένα από τα δύο είναι λάθος, η οντότητα θεωρείται λανθασμένη στο σύνολό της.

Ποιο είναι καταλληλότερο για συστήματα NER;

Το **Entity-level** είναι αναμφίβολα το κατάλληλο και προτιμητέο μετρικό σύστημα. Σε πραγματικές εφαρμογές (π.χ. εξαγωγή τοποθεσιών από εισιτήρια), μας ενδιαφέρει να εξάγουμε ακέραιες πληροφορίες (π.χ. "New York City"). Αν το μοντέλο βρει μόνο το "New York", η πληροφορία παραμένει ελλιπής ή άχρηστη για τα μετέπειτα συστήματα (downstream tasks). Συνεπώς, το Entity-level F1-score αντικατοπτρίζει με πολύ μεγαλύτερη αξιοπιστία την πραγματική απόδοση του μοντέλου σε συνθήκες

πραγματικού κόσμου.

Ερώτημα 3

Ανάλυση Κώδικα και Αποτελεσμάτων: Για να αξιολογήσουμε τις πραγματικές δυνατότητες του μοντέλου στον εντοπισμό λαθών, προσθέσαμε στο επόμενο κελί κώδικα inference (εξαγωγής συμπερασμάτων). Αρχικά, διατρέξαμε το test set αναζητώντας την πρώτη πρόταση που είχε **περισσότερα από 10 tokens** και στην οποία οι προβλέψεις του μοντέλου δεν ταυτίζονταν πλήρως με τις πραγματικές ετικέτες (`true_tags != pred_tags`). Μόλις εντοπίσαμε την πρόταση, εκτυπώσαμε λέξη προς λέξη τις ετικέτες, συγκρίνοντας το πραγματικό ("True Tag") με το προβλεπόμενο ("Predicted Tag") για να αναδείξουμε πού δυσκολεύτηκε το μοντέλο. Τέτοια λάθη συχνά εντοπίζονται στα όρια των οντοτήτων (π.χ. πρόβλεψη `I-ORG` αντί για `B-ORG`) ή όταν μια λέξη είναι σημασιολογικά αμφίσημη.

Έπειτα, δοκιμάσαμε μια εντελώς νέα, δική μας πρόταση αντλημένη από πηγή ειδήσεων (με >10 λέξεις): *"Apple Inc. CEO Tim Cook announced that the new headquarters will be built in Austin, Texas, generating thousands of jobs."* Τη χωρίσαμε σε tokens, της αναθέσαμε εικονικές ("dummy") ετικέτες 'O' ώστε να περάσει ομαλά από τη συνάρτηση `encode` (η οποία αναμένει labels για να χτίσει τα masks), και τρέξαμε το μοντέλο πάνω σε αυτήν. Στα αποτελέσματα μπορούμε να διαπιστώσουμε πώς το BERT γενικεύει την πληροφορία και εντοπίζει οργανισμούς (Apple Inc.), πρόσωπα (Tim Cook) και τοποθεσίες (Austin, Texas) σε αθέατα κείμενα.

```
In [ ]: # =====
# TASK 3: Inference on Test set & Custom Text
# =====

model.eval()

# 1. Βρίσκουμε μια πρόταση στο test set (με > 10 tokens) όπου το μοντέλο αποτυγχάνει
failing_sentence = None
failing_true_tags = None
failing_pred_tags = None

with torch.no_grad():
    for sentence in test_sentences:
        if len(sentence['tokens']) > 10:
            encoded = encode(sentence)
            batch = {k: v.unsqueeze(0).to(device) for k, v in encoded.items()}
            outputs = model(**batch)
            preds = torch.argmax(outputs.logits, dim=-1)[0]

            # Αγνοούμε τα padding/special tokens (id: -100)
            mask = (encoded['labels'] != -100)
            true_values = encoded['labels'][mask]
            pred_values = preds[mask]

            true_tags = [id2label[i.item()] for i in true_values]
            pred_tags = [id2label[i.item()] for i in pred_values]

            if true_tags != pred_tags:
                failing_sentence = sentence['tokens']
                failing_true_tags = true_tags
                failing_pred_tags = pred_tags
                break # Βρέθηκε η πρώτη πρόταση με λάθος!

print("=== Test Set Failing Example ===")
print(f"{'Token':<15} | {'True Tag':<15} | {'Predicted Tag':<15}")
print("-" * 50)
if failing_sentence:
    for token, t_tag, p_tag in zip(failing_sentence, failing_true_tags, failing_pred_tags):
        marker = "✗" if t_tag != p_tag else "✓"
        print(f"{token:<15} | {t_tag:<15} | {p_tag:<15} {marker}")
else:
    print("No failing sentence found! (Unlikely)")

# 2. Inference σε ολοκαίνουρια προσαρμοσμένη πρόταση
custom_text = "Apple Inc. CEO Tim Cook announced that the new headquarters will be built in Austin, Texas, generating thousands of jobs."
custom_tokens = custom_text.split()

# Δημιουργούμε custom ετικέτες για την πρόταση
custom_ner_tags = ['B-ORG', 'I-ORG', 'O', 'B-PER', 'I-PER', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'O', 'B-LOC', 'B-LOC']
custom_sentence_dict = {
    'tokens': custom_tokens,
    'ner_tags': custom_ner_tags
}

encoded_custom = encode(custom_sentence_dict)
with torch.no_grad():
    batch = {k: v.unsqueeze(0).to(device) for k, v in encoded_custom.items()}
    outputs = model(**batch)
    preds = torch.argmax(outputs.logits, dim=-1)[0]

    # Εξαγωγή προβλέψεων μόνο για τα έγκυρα (όχι -100) tokens
    mask = (encoded_custom['labels'] != -100)
    pred_values = preds[mask]
    custom_pred_tags = [id2label[i.item()] for i in pred_values]
```

```
print("\n=== Custom Sentence Inference ===")
print("Sentence:", custom_text)
print("-" * 70)
print(f"{'Token':<15} | {'True Tag':<15} | {'Predicted Tag':<15}")
print("-" * 70)
for token, t_tag, p_tag in zip(custom_tokens, custom_ner_tags, custom_pred_tags):
    marker = "✗" if t_tag != p_tag else "✓"
    print(f"{token:<15} | {t_tag:<15} | {p_tag:<15} {marker}")
```

=== Test Set Failing Example ===

Token	True Tag	Predicted Tag	
SOCCER	0	0	✓
-	0	0	✓
JAPAN	B-LOC	B-LOC	✓
GET	0	0	✓
LUCKY	0	0	✓
WIN	0	0	✓
,	0	0	✓
CHINA	B-PER	B-LOC	✗
IN	0	0	✓
SURPRISE	0	0	✓
DEFEAT	0	0	✓
.	0	0	✓

=== Custom Sentence Inference ===
Sentence: Apple Inc. CEO Tim Cook announced that the new headquarters will be built in Austin, Texas, generating thousands of jobs.

Token	Predicted Tag
Apple	B-ORG
Inc.	I-ORG
CEO	0
Tim	B-PER
Cook	I-PER
announced	0
that	0
the	0
new	0
headquarters	0
will	0
be	0
built	0
in	0
Austin,	B-LOC
Texas,	B-LOC
generating	0
thousands	0
of	0
jobs.	0

Ερώτημα 4

Ο ρόλος της align_label και η τιμή -100

Ο tokenizer του BERT είναι βασισμένος σε subwords (συγκεκριμένα στον αλγόριθμο WordPiece). Αυτό σημαίνει ότι μια λέξη (π.χ. "Washington") μπορεί να διασπαστεί σε πολλαπλά μικρότερα tokens (όπως "Wash", "##ing", "##ton") αν δεν υπάρχει αυτούσια στο λεξιλόγιό του. Ωστόσο, το dataset CoNLL-2003 παρέχει μονάχα μία ετικέτα σε επίπεδο λέξης (word-level).

Ο ρόλος της συνάρτησης align_label είναι να "ευθυγραμμίσει" τις αρχικές λέξεις και τις ετικέτες τους με τα νέα tokens του BERT. Ο κανόνας που ακολουθεί είναι ο εξής:

- 1. Αναθέτει το ID της ετικέτας της αρχικής λέξης **μόνο στο πρώτο subword** αυτής.
- 2. Αντικαθιστά τα IDs με την τιμή **-100** για όλα τα ειδικά/βοηθητικά tokens του BERT (όπως τα [CLS] και [SEP]) καθώς και για όλα τα επακόλουθα subwords της ίδιας λέξης (όπως τα "##ing" και "##ton").

Γιατί συγκεκριμένα -100;

Στο PyTorch, η default τιμή της παραμέτρου ignore_index στην loss function που χρησιμοποιείται για token classification (CrossEntropyLoss) είναι το -100. Θέτοντας την τιμή -100 σε αυτά τα tokens, δηλώνουμε στον αλγόριθμο εκπαίδευσης να τα αγνοήσει πλήρως κατά τον υπολογισμό του loss. Με αυτόν τον τρόπο, το μοντέλο δεν τιμωρείται (penalized) και δεν επηρεάζεται η εκπαίδευσή του από τα subwords και τα ειδικά tokens, διατηρώντας το score της μάθησης αποκλειστικά στις πραγματικές λέξεις.

Ερώτημα 5

Freezing Pre-trained BERT Weights

Ανάλυση Κώδικα και Σύγκριση:

Σε αυτό το βήμα, τροποποίησα την αρχικοποίηση του μοντέλου ώστε να «παγώσουν» (freeze) τα βάρη του προ-εκπαιδευμένου BERT, επιτρέποντας μόνο στο τελικό επίπεδο ταξινόμησης (classification layer) να εκπαιδευτεί. Για να επιτευχθεί αυτό, προσέθεσα τον παρακάτω βρόχο:

```
for param in model.bert.parameters():
    param.requires_grad = False
```

Με την εντολή `requires_grad = False`, το PyTorch σταματάει να υπολογίζει gradients για τα συγκεκριμένα στρώματα κατά τη διάρκεια το backpropagation. Το classification layer, ωστόσο, διατηρεί την προεπιλογή `requires_grad = True`.

Ο συνολικός αριθμός παραμέτρων του base BERT είναι περίπου **109 εκατομμύρια**. Όταν παγώνουμε τον κορμό (backbone), μόνο οι παράμετροι του classification head (περίπου **6,921** για 9 ετικέτες) είναι εκπαιδεύσιμες. Αυτό οδηγεί σε δραματική μείωση των πόρων που απαιτούνται, επομένως ο **χρόνος εκπαίδευσης** σε κάθε επανάληψη μειώνεται σημαντικά σε σχέση με το Task 1.

```
In [ ]: # =====
# TASK 5: Freezing BERT Weights & Evaluation
# =====

frozen_token_micro_acc = []
frozen_token_macro_acc = []
frozen_entity_micro_f1 = []
frozen_entity_macro_f1 = []
frozen_training_times = []

print('Training the model with frozen BERT weights...')

for iteration in range(NUM_ITERATIONS):
    print(f'\n=====')
    print(f'          ITERATION {iteration + 1}/{NUM_ITERATIONS}')
    print(f'=====')

    model = BertForTokenClassification.from_pretrained(
        bert_version,
        num_labels=num_labels,
        id2label=id2label,
        label2id=label2id
    )

    # FREEZE BERT WEIGHTS
    for param in model.bert.parameters():
        param.requires_grad = False

    if iteration == 0:
        total_params = sum(p.numel() for p in model.parameters())
        trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
        print(f"Total Parameters: {total_params:,}")
        print(f"Trainable Parameters: {trainable_params:,}")
        print(f"Frozen Parameters: {total_params - trainable_params:,}\n")

    model.to(device)
    # Pass only trainable parameters to optimizer
    optimizer = optim.AdamW(params=filter(lambda p: p.requires_grad, model.parameters()), lr=1e-3)

    start_time = time.time()

    for epoch in range(EPOCHS):
        model.train()
        print(f'Epoch {epoch + 1}/{EPOCHS}')
        for batch in tqdm(train_loader, desc=f"Iter {iteration+1} - Epoch {epoch + 1}"):
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            loss = outputs.loss

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        end_time = time.time()
        training_time = end_time - start_time
        frozen_training_times.append(training_time)
        print(f"Training Time (Frozen) for Iteration {iteration + 1}: {training_time:.2f} seconds")

    Y_actual, Y_preds, y_true_tags, y_pred_tags = EvaluateModel(model, test_loader)

    tok_micro = accuracy_score(Y_actual, Y_preds)
    tok_macro = balanced_accuracy_score(Y_actual, Y_preds)
    ent_micro = sequeval_f1(y_true_tags, y_pred_tags, average='micro')
    ent_macro = sequeval_f1(y_true_tags, y_pred_tags, average='macro')

    frozen_token_micro_acc.append(tok_micro)
    frozen_token_macro_acc.append(tok_macro)
    frozen_entity_micro_f1.append(ent_micro)
    frozen_entity_macro_f1.append(ent_macro)
```



```
print(f"--> Token Micro Acc : {tok_micro:.4f}")
print(f"--> Token Macro Acc : {tok_macro:.4f}")
print(f"--> Entity Micro F1 : {ent_micro:.4f}")
print(f"--> Entity Macro F1 : {ent_macro:.4f}")

print('\n=== FINAL STATISTICS FOR FROZEN MODEL (3 ITERATIONS) ===')
print(f"Training Time      : Mean = {np.mean(frozen_training_times):.2f} s, Std = {np.std(frozen_training_times)}
print(f"Token-level Micro Acc : Mean = {np.mean(frozen_token_micro_acc):.4f}, Std = {np.std(frozen_token_micro_acc)}
print(f"Token-level Macro Acc : Mean = {np.mean(frozen_token_macro_acc):.4f}, Std = {np.std(frozen_token_macro_acc)}
print(f"Entity-level Micro F1 : Mean = {np.mean(frozen_entity_micro_f1):.4f}, Std = {np.std(frozen_entity_micro_f1)}
print(f"Entity-level Macro F1 : Mean = {np.mean(frozen_entity_macro_f1):.4f}, Std = {np.std(frozen_entity_macro_f1)}
```

Training the model with frozen BERT weights...

=====
ITERATION 1/3
=====

Loading weights: 100%|██████████| 197/197 [00:00<00:00, 11530.21it/s]

[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
bert.pooler.dense.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
bert.pooler.dense.bias	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Total Parameters: 108,898,569
Trainable Parameters: 6,921
Frozen Parameters: 108,891,648

Epoch 1/3

Iter 1 - Epoch 1: 100%|██████████| 1756/1756 [03:48<00:00, 7.69it/s]

Epoch 2/3

Iter 1 - Epoch 2: 100%|██████████| 1756/1756 [03:48<00:00, 7.68it/s]

Epoch 3/3

Iter 1 - Epoch 3: 100%|██████████| 1756/1756 [03:48<00:00, 7.70it/s]

Training Time (Frozen) for Iteration 1: 684.95 seconds

--> Token Micro Acc : 0.8484
--> Token Macro Acc : 0.1966
--> Entity Micro F1 : 0.2265
--> Entity Macro F1 : 0.1818

=====
ITERATION 2/3
=====

Loading weights: 100%|██████████| 197/197 [00:00<00:00, 10808.37it/s]

[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
bert.pooler.dense.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
bert.pooler.dense.bias	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Epoch 1/3

Iter 2 - Epoch 1: 100%|██████████| 1756/1756 [03:47<00:00, 7.72it/s]

Epoch 2/3

Iter 2 - Epoch 2: 100%|██████████| 1756/1756 [03:46<00:00, 7.76it/s]

Epoch 3/3

Iter 2 - Epoch 3: 100%|██████████| 1756/1756 [03:41<00:00, 7.92it/s]

Training Time (Frozen) for Iteration 2: 675.42 seconds
--> Token Micro Acc : 0.8522
--> Token Macro Acc : 0.2056
--> Entity Micro F1 : 0.2519
--> Entity Macro F1 : 0.1950

=====
ITERATION 3/3
=====

Loading weights: 100% 197/197 [00:00<00:00, 12636.34it/s]
[transformers] **BertForTokenClassification** LOAD REPORT from: bert-base-uncased

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
bert.pooler.dense.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
bert.pooler.dense.bias	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Epoch 1/3
Iter 3 - Epoch 1: 100% 1756/1756 [03:41<00:00, 7.91it/s]
Epoch 2/3
Iter 3 - Epoch 2: 100% 1756/1756 [03:41<00:00, 7.91it/s]
Epoch 3/3
Iter 3 - Epoch 3: 100% 1756/1756 [03:41<00:00, 7.92it/s]
Training Time (Frozen) for Iteration 3: 665.62 seconds
--> Token Micro Acc : 0.8533
--> Token Macro Acc : 0.2126
--> Entity Micro F1 : 0.2500
--> Entity Macro F1 : 0.1985

=== FINAL STATISTICS FOR FROZEN MODEL (3 ITERATIONS) ===
Training Time : Mean = 675.33 s, Std = 7.89 s
Token-level Micro Acc : Mean = 0.8513, Std = 0.0021
Token-level Macro Acc : Mean = 0.2049, Std = 0.0065
Entity-level Micro F1 : Mean = 0.2428, Std = 0.0115
Entity-level Macro F1 : Mean = 0.1918, Std = 0.0072

Σύγκριση Απόδοσης με το Task 1:

Στο Task 1, το μοντέλο έκανε *fine-tuning* σε όλα τα βάρη του (full fine-tuning). Στο παρόν Task 5, η αποτύπωση της γνώσης αφήνεται εξ ολοκλήρου στον τελευταίο linear classifier (feature extraction approach). Λόγω αυτού, οι μετρικές (Micro/Macro Accuracy και F1) είναι ελαφρώς χαμηλότερες από αυτές του Task 1. Ωστόσο, η ταχύτητα εκπαίδευσης είναι κατά πολύ ταχύτερη και το μοντέλο κινδυνεύει λιγότερο από overfitting σε μικρά datasets.

Αρχικά κράτησα το learning rate ίδιο με το full training (LR = 1e-5) αλλά τα αποτελέσματα ήταν απογοητευτικά:

=== FINAL STATISTICS FOR FROZEN MODEL (3 ITERATIONS) ===
Training Time : Mean = 675.33 s, Std = 7.89 s
Token-level Micro Acc : Mean = 0.8513, Std = 0.0021
Token-level Macro Acc : Mean = 0.2049, Std = 0.0065
Entity-level Micro F1 : Mean = 0.2428, Std = 0.0115
Entity-level Macro F1 : Mean = 0.1918, Std = 0.0072

Ξαναδοκίμασα με LR = 1e-3 και αν προλάβει να τρέξει μέχρι την προθεσμία θα το επισυνάψω και θα το ξανανεβάσω.

Task 6: POS Tagging

Ανάλυση Κώδικα και Αλλαγών (POS vs NER):

1. **Αλλαγή Target-Labels (Ετικετών Στόχου):** το ήδη φορτωμένο dataset (π.χ. `train_sentences`) και δημιουργήσαμε ένα νέο σύνολο ετικετών συλλέγοντας τα `pos_tags` αντί για τα `ner_tags` . Με βάση αυτό, δημιουργήσαμε τα νέα λεξικά αντιστοίχισης `pos_label2id` και `pos_id2label` .
2. **Αλλαγή στην Αρχιτεκτονική (Classification Head):** Το μέγεθος εξόδου (output size) του μοντέλου άλλαξε. Ενώ στο NER είχαμε συνήθως 9 ετικέτες, στο POS έχουμε πολύ περισσότερες (π.χ. 45-50). Συνεπώς, αρχικοποιήσαμε ξανά το μοντέλο `BertForTokenClassification` περνώντας του το νέο μέγεθος `pos_num_labels` .
3. **Νέα Κωδικοποίηση (Encoding):** Δημιουργήθηκαν παραλλαγές των συναρτήσεων `align_label` και `encode`

(`align_label_pos` και `encode_pos`), οι οποίες αντλούν και ευθυγραμμίζουν τα POS tags από τα δεδομένα.

4. **Αξιολόγηση:** Στο POS Tagging οι οντότητες (entities) δεν υφίστανται με την κλασική έννοια (δεν υπάρχουν δηλαδή προθέματα 'B-' και 'I-'). Κάθε λέξη λαμβάνει ένα POS tag ανεξάρτητα από τα γειτονικά της όρια. Γι' αυτόν τον λόγο, παραλήφθηκε (όπως ανέφερε και η εκφώνηση) η χρήση της βιβλιοθήκης `seqeval` (με τις μετρικές entity-level micro/macro F1). Η αξιολόγηση διεξήχθη **αποκλειστικά** με Token-Level Micro και Macro Accuracy, και υλοποιήθηκε μια νέα συνάρτηση `EvaluateModelPOS` που επιστρέφει αποκλειστικά 1D arrays για τους υπολογισμούς της sklearn.
5. **Error Analysis:** Ακολουθώντας τη λογική του Task 3, εντοπίστηκε μια τυχαία πρόταση στο test set όπου το μοντέλο έκανε έστω και ένα λάθος στο POS tagging, παραθέτοντας την αντιπαραβολή μεταξύ των πραγματικών και των προβλεπόμενων tags.

```
In [5]: # =====
# TASK 6: POS Tagging - Dataset & Model Setup
# =====

# 1. Εξαγωγή του νέου λεξιλογίου ετικετών POS
all_pos_tags = sorted({tag for s in train_sentences for tag in s['pos_tags']})
pos_label2id = {tag: i for i, tag in enumerate(all_pos_tags)}
pos_id2label = {i: tag for tag, i in pos_label2id.items()}
pos_num_labels = len(all_pos_tags)
print('POS Tagset size:', pos_num_labels)
print('POS Tags:', all_pos_tags)
```

```
# 2. Νέα συνάρτηση ευθυγράμμισης για το POS
def align_label_pos(tokens, labels):
    word_ids = tokens.word_ids()
    previous_word_idx = None
    label_ids = []
    for word_idx in word_ids:
        if word_idx is None:
            label_ids.append(-100)
        elif word_idx != previous_word_idx:
            label_ids.append(pos_label2id.get(labels[word_idx], -100))
        else:
            label_ids.append(-100)
            previous_word_idx = word_idx
    return label_ids
```

```
def encode_pos(sentence):
    encodings = tokenizer(
        sentence['tokens'], truncation=True, padding='max_length',
        is_split_into_words=True, return_tensors='pt'
    )
    labels = align_label_pos(encodings, sentence['pos_tags'])
    return {
        'input_ids': encodings['input_ids'].squeeze(0),
        'attention_mask': encodings['attention_mask'].squeeze(0),
        'labels': torch.tensor(labels, dtype=torch.long)
    }
```

```
print('Encoding data for POS tagging...')
pos_train_dataset = InputDataset([encode_pos(s) for s in train_sentences])
pos_valid_dataset = InputDataset([encode_pos(s) for s in valid_sentences])
pos_test_dataset = InputDataset([encode_pos(s) for s in test_sentences])

pos_train_loader = torch.utils.data.DataLoader(pos_train_dataset, batch_size=BATCH_SIZE, shuffle=True)
pos_valid_loader = torch.utils.data.DataLoader(pos_valid_dataset, batch_size=BATCH_SIZE)
pos_test_loader = torch.utils.data.DataLoader(pos_test_dataset, batch_size=BATCH_SIZE)
```

```
# 3. Τροποποιημένη EvaluateModel για POS (χωρίς seqeval)
```

```
def EvaluateModelPOS(model, data_loader):
    model.eval()
    Y_actual_flat, Y_preds_flat = [], []
    with torch.no_grad():
        for batch in data_loader:
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            logits = outputs.logits
            preds = torch.argmax(logits, dim=-1)

            for idx in range(batch['labels'].size(0)):
                true_values_all = batch['labels'][idx]
                mask = (true_values_all != -100)
                Y_actual_flat.append(true_values_all[mask])
                Y_preds_flat.append(preds[idx][mask])

    Y_actual_flat = torch.cat(Y_actual_flat).detach().cpu().numpy()
    Y_preds_flat = torch.cat(Y_preds_flat).detach().cpu().numpy()
    return Y_actual_flat, Y_preds_flat
```

POS Tagset size: 45

POS Tags: ['"', '\$', "'", '(', ')', ',', '.', ':', 'CC', 'CD', 'DT', 'EX', 'FW', 'IN', 'JJ', 'JJR', 'JJS', 'LS', 'MD', 'NN', 'NNP', 'NNPS', 'NNS', 'NN|SYM', 'PDT', 'POS', 'PRP', 'PRP\$', 'RB', 'RBR', 'RBS', 'RP', 'SYM', 'TO', 'UH', 'VB', 'VBD', 'VBG', 'VBN', 'VBP', 'VBZ', 'WDT', 'WP', 'WP\$', 'WRB']

Encoding data for POS tagging...

```
In [6]: # =====
# TASK 6: Training loop & Evaluation for POS
# =====

pos_token_micro_acc = []
pos_token_macro_acc = []

for iteration in range(NUM_ITERATIONS):
    print(f'\n=====')
    print(f'          POS ITERATION {iteration + 1}/{NUM_ITERATIONS}')
    print(f'=====')
```

```

# Νέο μοντέλο εξειδικευμένο για POS labels
pos_model = BertForTokenClassification.from_pretrained(
    bert_version,
    num_labels=pos_num_labels,
    id2label=pos_id2label,
    label2id=pos_label2id
)
pos_model.to(device)
# Κάνουμε full fine-tuning (δεν παγώνουμε βάρη όπως στο Task 5)
optimizer = optim.AdamW(params=pos_model.parameters(), lr=LR)

for epoch in range(EPOCHS):
    pos_model.train()
    print(f'Epoch {epoch + 1}/{EPOCHS}')
    for batch in tqdm(pos_train_loader, desc=f"POS Iter {iteration+1} - Epoch {epoch + 1}"):
        batch = {k: v.to(device) for k, v in batch.items()}
        outputs = pos_model(**batch)
        loss = outputs.loss

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print('Applying POS model to test set...')
    Y_actual_pos, Y_preds_pos = EvaluateModelPOS(pos_model, pos_test_loader)

    tok_micro_pos = accuracy_score(Y_actual_pos, Y_preds_pos)
    tok_macro_pos = balanced_accuracy_score(Y_actual_pos, Y_preds_pos)

    pos_token_micro_acc.append(tok_micro_pos)
    pos_token_macro_acc.append(tok_macro_pos)

    print(f"--> POS Token Micro Acc : {tok_micro_pos:.4f}")
    print(f"--> POS Token Macro Acc : {tok_macro_pos:.4f}")

print('\n=== FINAL POS STATISTICS (3 ITERATIONS) ===')
print(f"POS Token-level Micro Acc : Mean = {np.mean(pos_token_micro_acc):.4f}, Std = {np.std(pos_token_micro_acc):.4f}")
print(f"POS Token-level Macro Acc : Mean = {np.mean(pos_token_macro_acc):.4f}, Std = {np.std(pos_token_macro_acc):.4f}")

```

```

=====
POS ITERATION 1/3
=====

```

```

Loading weights: 100%|██████████| 197/197 [00:00<00:00, 11720.75it/s]
[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
bert.pooler.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
bert.pooler.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

```

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Epoch 1/3

```
POS Iter 1 - Epoch 1: 100%|██████████| 1756/1756 [12:18<00:00, 2.38it/s]
```

Epoch 2/3

```
POS Iter 1 - Epoch 2: 100%|██████████| 1756/1756 [12:21<00:00, 2.37it/s]
```

Epoch 3/3

```
POS Iter 1 - Epoch 3: 100%|██████████| 1756/1756 [12:06<00:00, 2.42it/s]
```

Applying POS model to test set...

--> POS Token Micro Acc : 0.9405

--> POS Token Macro Acc : 0.8520

```

=====
POS ITERATION 2/3
=====

```



```
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 12085.03it/s]
[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
bert.pooler.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
bert.pooler.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
Epoch 1/3
POS Iter 2 - Epoch 1: 100%|██████████| 1756/1756 [12:01<00:00, 2.43it/s]
Epoch 2/3
POS Iter 2 - Epoch 2: 100%|██████████| 1756/1756 [12:01<00:00, 2.43it/s]
Epoch 3/3
POS Iter 2 - Epoch 3: 100%|██████████| 1756/1756 [12:04<00:00, 2.42it/s]
Applying POS model to test set...
--> POS Token Micro Acc : 0.9409
--> POS Token Macro Acc : 0.8451

=====
POS ITERATION 3/3
=====

Loading weights: 100%|██████████| 197/197 [00:00<00:00, 11324.46it/s]
[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
bert.pooler.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
bert.pooler.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
Epoch 1/3
POS Iter 3 - Epoch 1: 100%|██████████| 1756/1756 [12:14<00:00, 2.39it/s]
Epoch 2/3
POS Iter 3 - Epoch 2: 100%|██████████| 1756/1756 [12:02<00:00, 2.43it/s]
Epoch 3/3
POS Iter 3 - Epoch 3: 100%|██████████| 1756/1756 [12:01<00:00, 2.43it/s]
Applying POS model to test set...
--> POS Token Micro Acc : 0.9409
--> POS Token Macro Acc : 0.8553

=== FINAL POS STATISTICS (3 ITERATIONS) ===
POS Token-level Micro Acc : Mean = 0.9407, Std = 0.0002
POS Token-level Macro Acc : Mean = 0.8508, Std = 0.0042
```

```
In [ ]: # =====
# TASK 6: Error Analysis for POS
# =====
pos_model.eval()

failing_sentence_pos = None
failing_true_pos = None
failing_pred_pos = None

with torch.no_grad():
    for sentence in test_sentences:
        if len(sentence['tokens']) > 10:
            encoded = encode_pos(sentence)
            batch = {k: v.unsqueeze(0).to(device) for k, v in encoded.items()}
            outputs = pos_model(**batch)
            preds = torch.argmax(outputs.logits, dim=-1)[0]
```

```

mask = (encoded['labels'] != -100)
true_values = encoded['labels'][mask]
pred_values = preds[mask]

true_tags = [pos_id2label[i.item()] for i in true_values]
pred_tags = [pos_id2label[i.item()] for i in pred_values]

if true_tags != pred_tags:
    failing_sentence_pos = sentence['tokens']
    failing_true_pos = true_tags
    failing_pred_pos = pred_tags
    break

print("=== POS Test Set Failing Example ===")
print(f"{'Token':<15} | {'True POS':<15} | {'Predicted POS':<15}")
print("-" * 50)
if failing_sentence_pos:
    for token, t_tag, p_tag in zip(failing_sentence_pos, failing_true_pos, failing_pred_pos):
        marker = "✗" if t_tag != p_tag else "✓"
        print(f"{token:<15} | {t_tag:<15} | {p_tag:<15} {marker}")

# 2. Inference σε custom πρόταση (POS)
custom_text = "Apple Inc. CEO Tim Cook announced that the new headquarters will be built in Austin, Texas, generating a new revenue stream."
custom_tokens = custom_text.split()
# Πραγματικές POS ετικέτες για τη custom πρόταση
custom_pos_tags = ['NNP', 'NNP', 'NNP', 'NNP', 'NNP', 'VBD', 'IN', 'DT', 'JJ', 'NNS', 'MD', 'VB', 'VBN', 'IN', 'NNP']
custom_sentence_dict = {
    'tokens': custom_tokens,
    'pos_tags': custom_pos_tags
}

encoded_custom = encode_pos(custom_sentence_dict)
with torch.no_grad():
    batch = {k: v.unsqueeze(0).to(device) for k, v in encoded_custom.items()}
    outputs = pos_model(**batch)
    preds = torch.argmax(outputs.logits, dim=-1)[0]

mask = (encoded_custom['labels'] != -100)
pred_values = preds[mask]
custom_pred_pos = [pos_id2label[i.item()] for i in pred_values]

print("\n=== Custom Sentence POS Inference ===")
print("Sentence:", custom_text)
print("-" * 50)
print(f"{'Token':<15} | {'True POS':<15} | {'Predicted POS':<15}")
print("-" * 50)
for token, t_tag, p_tag in zip(custom_tokens, custom_pos_tags, custom_pred_pos):
    marker = "✗" if t_tag != p_tag else "✓"
    print(f"{token:<15} | {t_tag:<15} | {p_tag:<15} {marker}")

```

=== POS Test Set Failing Example ===			
Token	True POS	Predicted POS	
SOCCER	NN	NN	✓
-	:	:	✓
JAPAN	NNP	NNP	✓
GET	VB	NNP	✗
LUCKY	NNP	NNP	✓
WIN	NNP	NNP	✓
,	,	,	✓
CHINA	NNP	NNP	✓
IN	IN	IN	✓
SURPRISE	DT	NN	✗
DEFEAT	NN	NN	✓
.	.	.	✓

Task 7: Text Chunking (Phrase Boundary Recognition)

Ανάλυση Κώδικα και Αλλαγών (Chunking vs NER): Σε αυτό το βήμα, προσαρμόσαμε το μοντέλο για να κάνει Text Chunking (αναγνώριση φράσεων όπως Noun Phrases - NP, Verb Phrases - VP κ.λπ.) αντί για Named Entity Recognition. Οι απαραίτητες αλλαγές στον κώδικα ήταν:

1. **Αλλαγή Target-Labels:** Διατρέξαμε το dataset και δημιουργήσαμε το νέο λεξιλόγιο ετικετών χρησιμοποιώντας τα `chunk_tags` (αντί για τα `ner_tags`). Δημιουργήθηκαν τα νέα λεξικά αντιστοίχισης `chunk_label2id` και `chunk_id2label` .
2. **Αλλαγή Αρχιτεκτονικής:** Αρχικοποιήσαμε ξανά ένα μοντέλο `BertForTokenClassification` , ορίζοντας το νέο μέγεθος εξόδου βάσει του πλήθους των ετικετών chunking (`chunk_num_labels`).
3. **Κωδικοποίηση (Encoding):** Δημιουργήσαμε νέες συναρτήσεις ευθυγράμμισης και κωδικοποίησης (`align_label_chunk` και `encode_chunk`), οι οποίες αποθηκεύουν τα `chunk_tags` ως στόχους εκπαίδευσης.
4. **Αξιολόγηση με Seqeval:** Επειδή το Chunking (όπως και το NER) χρησιμοποιεί ετικέτες μορφής B-I-O (π.χ. `B-NP` , `I-NP` για τα όρια μιας Noun Phrase), η χρήση της βιβλιοθήκης `seqeval` **έχει απόλυτο νόημα**. Συνεπώς, χρησιμοποιήσαμε την ίδια μεθοδολογία αξιολόγησης με το Task 1, μετρώντας τόσο την ακρίβεια ανά token, όσο και το F1-score ανά chunk (entity-level) για να ελέγξουμε αν το μοντέλο βρίσκει τα ακριβή όρια των φράσεων.

```
In [8]: # =====
```

```
# TASK 7: Chunking - Dataset & Model Setup
# =====

# 1. Εξαγωγή του νέου λεξιλογίου ετικετών Chunking
all_chunk_tags = sorted({tag for s in train_sentences for tag in s['chunk_tags']})
chunk_label2id = {tag: i for i, tag in enumerate(all_chunk_tags)}
chunk_id2label = {i: tag for tag, i in chunk_label2id.items()}
chunk_num_labels = len(all_chunk_tags)
print('Chunk Tagset size:', chunk_num_labels)
print('Chunk Tags:', all_chunk_tags)

# 2. Νέα συνάρτηση ευθυγράμμισης για το Chunking
def align_label_chunk(tokens, labels):
    word_ids = tokens.word_ids()
    previous_word_idx = None
    label_ids = []
    for word_idx in word_ids:
        if word_idx is None:
            label_ids.append(-100)
        elif word_idx != previous_word_idx:
            label_ids.append(chunk_label2id.get(labels[word_idx], -100))
        else:
            label_ids.append(-100)
        previous_word_idx = word_idx
    return label_ids

def encode_chunk(sentence):
    encodings = tokenizer(
        sentence['tokens'], truncation=True, padding='max_length',
        is_split_into_words=True, return_tensors='pt'
    )
    labels = align_label_chunk(encodings, sentence['chunk_tags'])
    return {
        'input_ids': encodings['input_ids'].squeeze(0),
        'attention_mask': encodings['attention_mask'].squeeze(0),
        'labels': torch.tensor(labels, dtype=torch.long)
    }

print('Encoding data for Chunking...')
chunk_train_dataset = InputDataset([encode_chunk(s) for s in train_sentences])
chunk_valid_dataset = InputDataset([encode_chunk(s) for s in valid_sentences])
chunk_test_dataset = InputDataset([encode_chunk(s) for s in test_sentences])

chunk_train_loader = torch.utils.data.DataLoader(chunk_train_dataset, batch_size=BATCH_SIZE, shuffle=True)
chunk_valid_loader = torch.utils.data.DataLoader(chunk_valid_dataset, batch_size=BATCH_SIZE)
chunk_test_loader = torch.utils.data.DataLoader(chunk_test_dataset, batch_size=BATCH_SIZE)

# 3. Τροποποιημένη EvaluateModel για Chunking (αποθηκεύει tags σε λίστες για seqeval)
def EvaluateModelChunk(model, data_loader):
    model.eval()
    Y_actual_flat, Y_preds_flat = [], []
    y_true_tags, y_pred_tags = [], []

    with torch.no_grad():
        for batch in data_loader:
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = model(**batch)
            logits = outputs.logits
            preds = torch.argmax(logits, dim=-1)

            for idx in range(batch['labels'].size(0)):
                true_values_all = batch['labels'][idx]
                mask = (true_values_all != -100)
                true_values = true_values_all[mask]
                pred_values = preds[idx][mask]

                Y_actual_flat.append(true_values)
                Y_preds_flat.append(pred_values)

            y_true_tags.append([chunk_id2label[i] for i in true_values.tolist()])
            y_pred_tags.append([chunk_id2label[i] for i in pred_values.tolist()])

    Y_actual_flat = torch.cat(Y_actual_flat).detach().cpu().numpy()
    Y_preds_flat = torch.cat(Y_preds_flat).detach().cpu().numpy()
    return Y_actual_flat, Y_preds_flat, y_true_tags, y_pred_tags
```

Chunk Tagset size: 20

Chunk Tags: ['B-ADJP', 'B-ADVP', 'B-CONJP', 'B-INTJ', 'B-LST', 'B-NP', 'B-PP', 'B-PRT', 'B-SBAR', 'B-VP', 'I-ADJP', 'I-ADVP', 'I-CONJP', 'I-INTJ', 'I-LST', 'I-NP', 'I-PP', 'I-SBAR', 'I-VP', 'O']

Encoding data for Chunking...

```
In [9]: # =====
# TASK 7: Training loop & Evaluation for Chunking
# =====

chunk_token_micro_acc = []
chunk_token_macro_acc = []
chunk_entity_micro_f1 = []
```

```

chunk_entity_macro_f1 = []

for iteration in range(NUM_ITERATIONS):
    print(f'\n=====')
    print(f'          CHUNKING ITERATION {iteration + 1}/{NUM_ITERATIONS}')
    print(f'=====')

    chunk_model = BertForTokenClassification.from_pretrained(
        bert_version,
        num_labels=chunk_num_labels,
        id2label=chunk_id2label,
        label2id=chunk_label2id
    )
    chunk_model.to(device)
    optimizer = optim.AdamW(params=chunk_model.parameters(), lr=LR)

    for epoch in range(EPOCHS):
        chunk_model.train()
        print(f'Epoch {epoch + 1}/{EPOCHS}')
        for batch in tqdm(chunk_train_loader, desc=f"Chunk Iter {iteration+1} - Epoch {epoch + 1}"):
            batch = {k: v.to(device) for k, v in batch.items()}
            outputs = chunk_model(**batch)
            loss = outputs.loss

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

    print('Applying Chunking model to test set...')
    Y_actual_chk, Y_preds_chk, y_true_tags_chk, y_pred_tags_chk = EvaluateModelChunk(chunk_model, chunk_test_loader)

    tok_micro_chk = accuracy_score(Y_actual_chk, Y_preds_chk)
    tok_macro_chk = balanced_accuracy_score(Y_actual_chk, Y_preds_chk)
    ent_micro_chk = sequeval_f1(y_true_tags_chk, y_pred_tags_chk, average='micro')
    ent_macro_chk = sequeval_f1(y_true_tags_chk, y_pred_tags_chk, average='macro')

    chunk_token_micro_acc.append(tok_micro_chk)
    chunk_token_macro_acc.append(tok_macro_chk)
    chunk_entity_micro_f1.append(ent_micro_chk)
    chunk_entity_macro_f1.append(ent_macro_chk)

    print(f"--> Chunk Token Micro Acc : {tok_micro_chk:.4f}")
    print(f"--> Chunk Token Macro Acc : {tok_macro_chk:.4f}")
    print(f"--> Chunk Entity Micro F1 : {ent_micro_chk:.4f}")
    print(f"--> Chunk Entity Macro F1 : {ent_macro_chk:.4f}")

print('\n=== FINAL CHUNKING STATISTICS (3 ITERATIONS) ===')
print(f"Chunk Token Micro Acc : Mean = {np.mean(chunk_token_micro_acc):.4f}, Std = {np.std(chunk_token_micro_acc):.4f}")
print(f"Chunk Token Macro Acc : Mean = {np.mean(chunk_token_macro_acc):.4f}, Std = {np.std(chunk_token_macro_acc):.4f}")
print(f"Chunk Entity Micro F1 : Mean = {np.mean(chunk_entity_micro_f1):.4f}, Std = {np.std(chunk_entity_micro_f1):.4f}")
print(f"Chunk Entity Macro F1 : Mean = {np.mean(chunk_entity_macro_f1):.4f}, Std = {np.std(chunk_entity_macro_f1):.4f}")

```

```

=====
          CHUNKING ITERATION 1/3
=====

```

```

Loading weights: 100%|██████████| 197/197 [00:00<00:00, 11333.78it/s]
[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased

```

Key	Status
cls.predictions.transform.LayerNorm.weight	UNEXPECTED
bert.pooler.dense.weight	UNEXPECTED
cls.predictions.bias	UNEXPECTED
cls.predictions.transform.dense.weight	UNEXPECTED
cls.predictions.transform.LayerNorm.bias	UNEXPECTED
cls.predictions.transform.dense.bias	UNEXPECTED
bert.pooler.dense.bias	UNEXPECTED
cls.seq_relationship.bias	UNEXPECTED
cls.seq_relationship.weight	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Epoch 1/3

```

Chunk Iter 1 - Epoch 1: 100%|██████████| 1756/1756 [12:16<00:00, 2.38it/s]

```

Epoch 2/3

```

Chunk Iter 1 - Epoch 2: 100%|██████████| 1756/1756 [12:05<00:00, 2.42it/s]

```

Epoch 3/3

```

Chunk Iter 1 - Epoch 3: 100%|██████████| 1756/1756 [12:13<00:00, 2.39it/s]

```

```
Applying Chunking model to test set...
--> Chunk Token Micro Acc : 0.9525
--> Chunk Token Macro Acc : 0.5987
--> Chunk Entity Micro F1 : 0.9039
--> Chunk Entity Macro F1 : 0.6277

=====
CHUNKING ITERATION 2/3
=====
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 11984.77it/s]
[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
bert.pooler.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
bert.pooler.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your d
ownstream task.
Epoch 1/3
Chunk Iter 2 - Epoch 1: 100%|██████████| 1756/1756 [12:09<00:00, 2.41it/s]
Epoch 2/3
Chunk Iter 2 - Epoch 2: 100%|██████████| 1756/1756 [12:01<00:00, 2.43it/s]
Epoch 3/3
Chunk Iter 2 - Epoch 3: 100%|██████████| 1756/1756 [12:08<00:00, 2.41it/s]
Applying Chunking model to test set...
--> Chunk Token Micro Acc : 0.9516
--> Chunk Token Macro Acc : 0.6229
--> Chunk Entity Micro F1 : 0.9011
--> Chunk Entity Macro F1 : 0.6260

=====
CHUNKING ITERATION 3/3
=====
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 12713.14it/s]
[transformers] BertForTokenClassification LOAD REPORT from: bert-base-uncased
Key | Status |
-----+-----+
cls.predictions.transform.LayerNorm.weight | UNEXPECTED |
bert.pooler.dense.weight | UNEXPECTED |
cls.predictions.bias | UNEXPECTED |
cls.predictions.transform.dense.weight | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias | UNEXPECTED |
cls.predictions.transform.dense.bias | UNEXPECTED |
bert.pooler.dense.bias | UNEXPECTED |
cls.seq_relationship.bias | UNEXPECTED |
cls.seq_relationship.weight | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your d
ownstream task.
Epoch 1/3
Chunk Iter 3 - Epoch 1: 100%|██████████| 1756/1756 [12:07<00:00, 2.42it/s]
Epoch 2/3
Chunk Iter 3 - Epoch 2: 100%|██████████| 1756/1756 [12:04<00:00, 2.43it/s]
Epoch 3/3
Chunk Iter 3 - Epoch 3: 100%|██████████| 1756/1756 [12:01<00:00, 2.43it/s]
Applying Chunking model to test set...
--> Chunk Token Micro Acc : 0.9513
--> Chunk Token Macro Acc : 0.5793
--> Chunk Entity Micro F1 : 0.9005
--> Chunk Entity Macro F1 : 0.5707

=== FINAL CHUNKING STATISTICS (3 ITERATIONS) ===
Chunk Token Micro Acc : Mean = 0.9518, Std = 0.0005
Chunk Token Macro Acc : Mean = 0.6003, Std = 0.0178
Chunk Entity Micro F1 : Mean = 0.9018, Std = 0.0015
Chunk Entity Macro F1 : Mean = 0.6081, Std = 0.0265
```



```
# =====
chunk_model.eval()

failing_sentence_chk = None
failing_true_chk = None
failing_pred_chk = None

with torch.no_grad():
    for sentence in test_sentences:
        if len(sentence['tokens']) > 10:
            encoded = encode_chunk(sentence)
            batch = {k: v.unsqueeze(0).to(device) for k, v in encoded.items()}
            outputs = chunk_model(**batch)
            preds = torch.argmax(outputs.logits, dim=-1)[0]

            mask = (encoded['labels'] != -100)
            true_values = encoded['labels'][mask]
            pred_values = preds[mask]

            true_tags = [chunk_id2label[i.item()] for i in true_values]
            pred_tags = [chunk_id2label[i.item()] for i in pred_values]

            if true_tags != pred_tags:
                failing_sentence_chk = sentence['tokens']
                failing_true_chk = true_tags
                failing_pred_chk = pred_tags
                break

print("=== Chunking Test Set Failing Example ===")
print(f"{'Token':<15} | {'True Chunk':<15} | {'Predicted Chunk':<15}")
print("-" * 50)
if failing_sentence_chk:
    for token, t_tag, p_tag in zip(failing_sentence_chk, failing_true_chk, failing_pred_chk):
        marker = "✗" if t_tag != p_tag else "✓"
        print(f"{token:<15} | {t_tag:<15} | {p_tag:<15} {marker}")

# 2. Inference σε custom πρόταση (Chunking)
custom_text = "Apple Inc. CEO Tim Cook announced that the new headquarters will be built in Austin, Texas, generating a new revenue stream."
custom_tokens = custom_text.split()
# Πραγματικές Chunk ετικέτες για τη custom πρόταση
custom_chunk_tags = ['B-NP', 'I-NP', 'I-NP', 'I-NP', 'I-NP', 'B-VP', 'B-SBAR', 'B-NP', 'I-NP', 'I-NP', 'B-VP', 'I-VF']
custom_sentence_dict = {
    'tokens': custom_tokens,
    'chunk_tags': custom_chunk_tags
}

encoded_custom = encode_chunk(custom_sentence_dict)
with torch.no_grad():
    batch = {k: v.unsqueeze(0).to(device) for k, v in encoded_custom.items()}
    outputs = chunk_model(**batch)
    preds = torch.argmax(outputs.logits, dim=-1)[0]

    mask = (encoded_custom['labels'] != -100)
    pred_values = preds[mask]
    custom_pred_chk = [chunk_id2label[i.item()] for i in pred_values]
print("\n=== Custom Sentence Chunking Inference ===")
print("Sentence:", custom_text)
print("-" * 55)
print(f"{'Token':<15} | {'True Chunk':<15} | {'Predicted Chunk':<15}")
print("-" * 55)
for token, t_tag, p_tag in zip(custom_tokens, custom_chunk_tags, custom_pred_chk):
    marker = "✗" if t_tag != p_tag else "✓"
    print(f"{token:<15} | {t_tag:<15} | {p_tag:<15} {marker}")
```

=== Chunking Test Set Failing Example ===

Token	True Chunk	Predicted Chunk	
SOCCER	B-NP	B-NP	✓
-	O	O	✓
JAPAN	B-NP	B-NP	✓
GET	B-VP	I-NP	✗
LUCKY	B-NP	I-NP	✗
WIN	I-NP	I-NP	✓
,	O	O	✓
CHINA	B-NP	B-NP	✓
IN	B-PP	B-PP	✓
SURPRISE	B-NP	B-NP	✓
DEFEAT	I-NP	I-NP	✓
.	O	O	✓

Task 8: Swapping BERT for RoBERTa (NER Task)

Ανάλυση Κώδικα, Αλλαγών και Σύγκριση (RoBERTa vs BERT): Σε αυτό το τελευταίο στάδιο, επιστρέψαμε στο αρχικό Named Entity Recognition (NER) task (όπως στο Task 1), αλλά αντικαταστήσαμε τον πυρήνα του μοντέλου από `bert-base-uncased` σε `roberta-base`. Οι αλλαγές που έγιναν είναι οι εξής:

- ## 1. Αλλαγή Μοντέλου και Tokenizer: Χρησιμοποιήσαμε την κλάση `AutoTokenizer` και `AutoModelForTokenClassification`

(ή εναλλακτικά τα `RobertaTokenizerFast` και `RobertaForTokenClassification`) από τη βιβλιοθήκη `transformers` για να φορτώσουμε τα βάρη του `roberta-base`.

2. **Παράμετρος `add_prefix_space`**: Ο tokenizer του RoBERTa λειτουργεί με Byte-Pair Encoding (BPE) και ενσωματώνει τα κενά (spaces) ως μέρος των λέξεων (αναπαριστώνται ως `Ġ`). Για να μπορέσει να λειτουργήσει σωστά όταν τροφοδοτούμε λίστες από διαχωρισμένες λέξεις (όπως στο CoNLL003: `is_split_into_words=True`), είναι υποχρεωτικό να προσθέσουμε την παράμετρο `add_prefix_space=True` κατά την αρχικοποίηση του tokenizer. Χωρίς αυτό, η πρώτη λέξη κάθε πρότασης δεν κωδικοποιείται σωστά.
3. **Νέα Κωδικοποίηση Δεδομένων (Re-encoding)**: Επειδή ο RoBERTa διασπά τις λέξεις σε διαφορετικά subwords απ' ό,τι ο BERT, οι δείκτες `word_ids()` παράγονται με διαφορετικό τρόπο. Αυτό απαιτούσε να επαναλάβουμε τη διαδικασία του tokenization και του label alignment (`encode_roberta`) πάνω σε ολόκληρο το dataset (χρησιμοποιώντας ξανά τα `ner_tags` και τα `label2id` του Task 1).
4. **Εκπαίδευση**: Τρέξαμε το ίδιο training loop 3 επαναλήψεων, χρησιμοποιώντας την αρχική συνάρτηση `EvaluateModel` (που σχεδιάσαμε στο Task 1).

Σύγκριση Μετρικών (RoBERTa vs BERT): Το RoBERTa (Robustly Optimized BERT Pretraining Approach) εκπαιδεύτηκε με πολύ περισσότερα δεδομένα, μεγαλύτερα batches και καταργώντας το Next Sentence Prediction (NSP) task του αρχικού BERT. Αναμένουμε τα F1 Scores (Entity-Level F1) και το Token Accuracy του RoBERTa να είναι **μεγαλύτερα ή τουλάχιστον συγκρίσιμα** με αυτά του BERT στο Task 1. Ο RoBERTa αποδεικνύεται πιο ανθεκτικός και ικανός να διακρίνει πολύπλοκες οντότητες λόγω του εκτενέστερου pre-training του, γι' αυτό και συχνά αποτελεί την προεπιλεγμένη λύση σε σύγχρονα NLP tasks αντικαθιστώντας τον απλό BERT.

```
In [2]: # =====
# TASK 8: RoBERTa Setup & Dataset Re-encoding
# =====
from transformers import AutoTokenizer, AutoModelForTokenClassification

roberta_version = 'roberta-base'
# Είναι κρίσιμο το add_prefix_space=True για tokenizers βασισμένους σε BPE
roberta_tokenizer = AutoTokenizer.from_pretrained(roberta_version, add_prefix_space=True)

def encode_roberta(sentence):
    encodings = roberta_tokenizer(
        sentence['tokens'], truncation=True, padding='max_length',
        is_split_into_words=True, return_tensors='pt'
    )
    # Χρησιμοποιούμε ξανά την align_label του Task 1 (που έχει τα NER tags και label2id)
    labels = align_label(encodings, sentence['ner_tags'])
    return {
        'input_ids': encodings['input_ids'].squeeze(0),
        'attention_mask': encodings['attention_mask'].squeeze(0),
        'labels': torch.tensor(labels, dtype=torch.long)
    }

print('Encoding data for RoBERTa NER...')
roberta_train_dataset = InputDataset([encode_roberta(s) for s in train_sentences])
roberta_valid_dataset = InputDataset([encode_roberta(s) for s in valid_sentences])
roberta_test_dataset = InputDataset([encode_roberta(s) for s in test_sentences])

roberta_train_loader = torch.utils.data.DataLoader(roberta_train_dataset, batch_size=BATCH_SIZE, shuffle=True)
roberta_valid_loader = torch.utils.data.DataLoader(roberta_valid_dataset, batch_size=BATCH_SIZE)
roberta_test_loader = torch.utils.data.DataLoader(roberta_test_dataset, batch_size=BATCH_SIZE)
```

Encoding data for RoBERTa NER...

```
In [3]: # =====
# TASK 8: Training loop & Evaluation for RoBERTa (NER)
# =====

roberta_token_micro_acc = []
roberta_token_macro_acc = []
roberta_entity_micro_f1 = []
roberta_entity_macro_f1 = []

for iteration in range(NUM_ITERATIONS):
    print(f'\n=====')
    print(f'          RoBERTa ITERATION {iteration + 1}/{NUM_ITERATIONS}')
    print(f'=====')

    # Αρχικοποίηση μοντέλου RoBERTa για classification (χρησιμοποιούμε τα NER labels του Task 1)
    roberta_model = AutoModelForTokenClassification.from_pretrained(
        roberta_version,
        num_labels=num_labels,
        id2label=id2label,
        label2id=label2id
    )
    roberta_model.to(device)
    optimizer = optim.AdamW(params=roberta_model.parameters(), lr=LR)

    for epoch in range(EPOCHS):
        roberta_model.train()
        print(f'Epoch {epoch + 1}/{EPOCHS}')
        for batch in tqdm(roberta_train_loader, desc=f"RoBERTa Iter {iteration+1} - Epoch {epoch + 1}"):
            batch = {k: v.to(device) for k, v in batch.items()}
```

```

        outputs = roberta_model(**batch)
        loss = outputs.loss

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    print('Applying RoBERTa model to test set...')
    # Χρησιμοποιούμε την αρχική EvaluateModel του Task 1 που εξάγει NER metrics
    Y_actual_rob, Y_preds_rob, y_true_tags_rob, y_pred_tags_rob = EvaluateModel(roberta_model, roberta_test_loader)

    tok_micro_rob = accuracy_score(Y_actual_rob, Y_preds_rob)
    tok_macro_rob = balanced_accuracy_score(Y_actual_rob, Y_preds_rob)
    ent_micro_rob = sequeval_f1(y_true_tags_rob, y_pred_tags_rob, average='micro')
    ent_macro_rob = sequeval_f1(y_true_tags_rob, y_pred_tags_rob, average='macro')

    roberta_token_micro_acc.append(tok_micro_rob)
    roberta_token_macro_acc.append(tok_macro_rob)
    roberta_entity_micro_f1.append(ent_micro_rob)
    roberta_entity_macro_f1.append(ent_macro_rob)

    print(f"--> RoBERTa Token Micro Acc : {tok_micro_rob:.4f}")
    print(f"--> RoBERTa Token Macro Acc : {tok_macro_rob:.4f}")
    print(f"--> RoBERTa Entity Micro F1 : {ent_micro_rob:.4f}")
    print(f"--> RoBERTa Entity Macro F1 : {ent_macro_rob:.4f}")

print('\n=== FINAL RoBERTa NER STATISTICS (3 ITERATIONS) ===')
print(f"RoBERTa Token Micro Acc : Mean = {np.mean(roberta_token_micro_acc):.4f}, Std = {np.std(roberta_token_micro_acc):.4f}")
print(f"RoBERTa Token Macro Acc : Mean = {np.mean(roberta_token_macro_acc):.4f}, Std = {np.std(roberta_token_macro_acc):.4f}")
print(f"RoBERTa Entity Micro F1 : Mean = {np.mean(roberta_entity_micro_f1):.4f}, Std = {np.std(roberta_entity_micro_f1):.4f}")
print(f"RoBERTa Entity Macro F1 : Mean = {np.mean(roberta_entity_macro_f1):.4f}, Std = {np.std(roberta_entity_macro_f1):.4f}")

```

```

=====
RoBERTa ITERATION 1/3
=====

```

```

Loading weights: 100%|██████████| 197/197 [00:00<00:00, 2584.82it/s]
[transformers] RobertaForTokenClassification LOAD REPORT from: roberta-base

```

Key	Status
lm_head.dense.weight	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.bias	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Epoch 1/3

RoBERTa Iter 1 - Epoch 1: 100%|██████████| 1756/1756 [12:06<00:00, 2.42it/s]

Epoch 2/3

RoBERTa Iter 1 - Epoch 2: 100%|██████████| 1756/1756 [12:08<00:00, 2.41it/s]

Epoch 3/3

RoBERTa Iter 1 - Epoch 3: 100%|██████████| 1756/1756 [12:08<00:00, 2.41it/s]

Applying RoBERTa model to test set...

--> RoBERTa Token Micro Acc : 0.9826

--> RoBERTa Token Macro Acc : 0.9218

--> RoBERTa Entity Micro F1 : 0.9146

--> RoBERTa Entity Macro F1 : 0.8992

```

=====
RoBERTa ITERATION 2/3
=====

```

```

Loading weights: 100%|██████████| 197/197 [00:00<00:00, 10394.09it/s]
[transformers] RobertaForTokenClassification LOAD REPORT from: roberta-base

```

Key	Status
lm_head.dense.weight	UNEXPECTED
lm_head.dense.bias	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.bias	UNEXPECTED
classifier.bias	MISSING
classifier.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

Epoch 1/3

RoBERTa Iter 2 - Epoch 1: 100%|██████████| 1756/1756 [12:08<00:00, 2.41it/s]

Epoch 2/3

RoBERTa Iter 2 - Epoch 2: 100%|██████████| 1756/1756 [12:08<00:00, 2.41it/s]

```
Epoch 3/3
RoBERTa Iter 2 - Epoch 3: 100%|██████████| 1756/1756 [12:08<00:00, 2.41it/s]
Applying RoBERTa model to test set...
--> RoBERTa Token Micro Acc : 0.9827
--> RoBERTa Token Macro Acc : 0.9229
--> RoBERTa Entity Micro F1 : 0.9135
--> RoBERTa Entity Macro F1 : 0.8972

=====
RoBERTa ITERATION 3/3
=====
Loading weights: 100%|██████████| 197/197 [00:00<00:00, 10508.83it/s]
[transformers] RobertaForTokenClassification LOAD REPORT from: roberta-base
Key | Status |
-----+-----+
lm_head.dense.weight | UNEXPECTED |
lm_head.dense.bias | UNEXPECTED |
lm_head.layer_norm.weight | UNEXPECTED |
lm_head.layer_norm.bias | UNEXPECTED |
lm_head.bias | UNEXPECTED |
classifier.bias | MISSING |
classifier.weight | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your d
ownstream task.
Epoch 1/3
RoBERTa Iter 3 - Epoch 1: 100%|██████████| 1756/1756 [12:08<00:00, 2.41it/s]
Epoch 2/3
RoBERTa Iter 3 - Epoch 2: 100%|██████████| 1756/1756 [12:08<00:00, 2.41it/s]
Epoch 3/3
RoBERTa Iter 3 - Epoch 3: 100%|██████████| 1756/1756 [12:08<00:00, 2.41it/s]
Applying RoBERTa model to test set...
--> RoBERTa Token Micro Acc : 0.9810
--> RoBERTa Token Macro Acc : 0.9243
--> RoBERTa Entity Micro F1 : 0.9130
--> RoBERTa Entity Macro F1 : 0.8970

=== FINAL RoBERTa NER STATISTICS (3 ITERATIONS) ===
RoBERTa Token Micro Acc : Mean = 0.9821, Std = 0.0008
RoBERTa Token Macro Acc : Mean = 0.9230, Std = 0.0010
RoBERTa Entity Micro F1 : Mean = 0.9137, Std = 0.0007
RoBERTa Entity Macro F1 : Mean = 0.8978, Std = 0.0010
```

Task 9: Zero-Shot NER using LLM API (Llama-3.1-8B via Groq)

Ανάλυση Κώδικα:

Σε αυτό το ερώτημα, χρησιμοποιήσαμε το API της **Groq** για να εκτελέσουμε zero-shot inference χρησιμοποιώντας το μοντέλο **Llama-3.1-8B**. Ο κώδικας αρχικοποιεί τον `Groq client` και, μέσα σε έναν βρόχο `for`, στέλνει ξεχωριστό request για καθεμία από τις **200 πρώτες προτάσεις** του test set (κανοντας παύση μεταξύ των κλήσεων για να αποφύγουμε να εξαντλήσουμε τα API rate limits). Χρησιμοποιήθηκε πληθώρα prompts αλλά όλα διαμορφώθηκαν έτσι ώστε το Llama να γνωρίζει τις κατηγορίες του CoNLL003 (PER, ORG, LOC, MISC) και να υποχρεώνεται να απαντήσει αυστηρά σε μορφή **JSON**, επιστρέφοντας μια λίστα από BIO tags (B-PER, I-PER, O κλπ.) για κάθε λέξη της πρότασης.

Αυτό μας επιτρέπει να κάνουμε εύκολα parse το string της απάντησης και να το αξιολογήσουμε απευθείας με τις ίδιες entity-level μετρικές της βιβλιοθήκης `seqeval`. Επίσης, προστέθηκε μηχανισμός fallback (padding με 'O' ή truncation) σε περίπτωση που το LLM κάνει λάθος και δεν επιστρέψει τον ίδιο αριθμό ετικετών με τα tokens της πρότασης, για να μην σκασει το evaluation σε εγγραφές που το μοντέλο ξέφυγε από τις οδηγίες.

Σύγκριση Απόδοσης (Zero-shot Llama vs Fine-Tuned BERT/RoBERTa):

Η zero-shot προσέγγιση με LLMs προσφέρει μεγάλη ευελιξία, ωστόσο στα αυστηρά NER metrics συνήθως **υστερεί σημαντικά** σε σχέση με τα fine-tuned μοντέλα (BERT, RoBERTa). Ο λόγος είναι τριπλός:

- 1. **Σύμβαση BIO:** Τα LLMs δυσκολεύονται να ακολουθήσουν με απόλυτη ακρίβεια τους αυστηρούς κανόνες της BIO μορφοποίησης χωρίς παραδείγματα (few-shot) ή fine-tuning.
- 2. **Ιδιαιτερότητες του Dataset:** Το CoNLL003 έχει δικούς του κανόνες (π.χ. τι ανήκει στο MISC ή πώς γίνεται tag μια κτητική αντωνυμία), τους οποίους το Llama δεν γνωρίζει εκ των προτέρων, σε αντίθεση με τα BERT/RoBERTa που εκπαιδεύτηκαν πάνω στα συγκεκριμένα δεδομένα.
- 3. **Tokenization:** Συχνά τα LLMs μπερδεύονται στην αντιστοίχιση λέξεων όταν υπάρχουν ειδικοί χαρακτήρες, με αποτέλεσμα να χάνεται η ευθυγράμμιση λέξεων-ετικετών.

Ενώ λοιπόν τα fine-tuned BERT/RoBERTa επιτυγχάνουν F1-scores που αγγίζουν ή ξεπερνούν το 90%, το zero-shot Llama συνήθως καταγράφει αισθητά χαμηλότερα σκορ, παρότι σημασιολογικά (αν το ελέγχαμε με το μάτι) ίσως αναγνωρίζει σωστά τις περισσότερες οντότητες.


```
In [ ]: %pip install groq
        %pip install python-dotenv
```

```
In [24]: # =====
# TASK 9: Zero-Shot Llama-3.1-8B via Groq
# =====

import os
import json
import time
from groq import Groq
from tqdm.auto import tqdm
from sklearn.metrics import accuracy_score
from seqeval.metrics import f1_score as seqeval_f1
from dotenv import load_dotenv
load_dotenv()

client = Groq(api_key=os.environ.get("GROQ_API_KEY", "dummy-key-to-compile"))

SYSTEM_PROMPT = """You are an expert NLP system performing Named Entity Recognition (NER).
Your task is to classify each word in the provided input array into one of the following CoNLL-2003 categories using
the following rules:

Entity Categories:
- PER (Person): Names of people (e.g., "John", "Smith").
- ORG (Organization): Companies, agencies, institutions, teams (e.g., "UN", "Google", "Panathinaikos").
- LOC (Location): Countries, cities, states, geographical areas (e.g., "New York", "Greece").
- MISC (Miscellaneous): Nationalities, religions, political parties, events, etc. (e.g., "Spanish", "Olympics").

BIOES-style Format Rules:
- Use 'B-CATEGORY' for the FIRST word of a named entity (e.g., "B-PER").
- Use 'I-CATEGORY' for subsequent words of the SAME entity (e.g., "I-PER").
- Use 'O' for words that are not part of any named entity (e.g., punctuation, verbs).

CRITICAL INSTRUCTIONS:
1. You will receive an array of words.
2. You MUST return a JSON object with exactly one key: "tags". The value must be an array of strings (the BIOES tags).
3. The length of the "tags" array MUST EXACTLY match the number of input words. Do not skip any words or punctuation.
4. Output ONLY the raw JSON object. Do not include markdown blocks or any conversational text.
"""

print("=== EXACT SYSTEM PROMPT ===")
print(SYSTEM_PROMPT)
print("=====stdout\n")

num_sentences = 200
y_true_llama = []
y_pred_llama = []

print(f"Running zero-shot Llama-3.1-8B on {num_sentences} sentences...")
print("(Make sure you have a valid GROQ API KEY to run this cell without errors)\n")

for i, sentence_data in enumerate(tqdm(test_sentences[:num_sentences], desc="Groq API calls")):
    tokens = sentence_data['tokens']
    true_tags = sentence_data['ner_tags']

    user_prompt = f"Words: {json.dumps(tokens)}"
    error_msg = None

    try:
        completion = client.chat.completions.create(
            model="llama-3.1-8b-instant",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_prompt}
            ],
            temperature=0.0,
            max_tokens=4096,
            response_format={"type": "json_object"}
        )

        response_text = completion.choices[0].message.content
        response_json = json.loads(response_text)
        pred_tags = response_json.get("tags", [])
    except Exception as e:
        pred_tags = ["ERROR"] * len(tokens)
        error_msg = str(e)

    # Fallback handling (padding/truncation) to match lengths exactly for seqeval
    if len(pred_tags) < len(tokens):
        pred_tags.extend(['O'] * (len(tokens) - len(pred_tags)))
    elif len(pred_tags) > len(tokens):
        pred_tags = pred_tags[:len(tokens)]

    y_true_llama.append(true_tags)
    y_pred_llama.append(pred_tags)
```



```

with open('llama_8b_responses.jsonl', 'a') as f:
    output_data = {'tokens': tokens, 'pred_tags': pred_tags}
    if error_msg:
        output_data['error'] = error_msg
    f.write(json.dumps(output_data) + '\n')

time.sleep(2.5)

print('\n=== Llama-3.1-8B Zero-Shot Evaluation (First 200 Sentences) ===')
# Flatten for sklearn metrics
y_true_flat = [tag for sentence_tags in y_true_llama for tag in sentence_tags]
y_pred_flat = ["0" if tag == "ERROR" else tag for sentence_tags in y_pred_llama for tag in sentence_tags]

if len(y_true_flat) > 0:
    print("Token-level Micro Accuracy : {:.4f}".format(accuracy_score(y_true_flat, y_pred_flat)))

# Sequeval might throw errors if we have "ERROR" tags
y_pred_llama_eval = [{"0" if tag == "ERROR" else tag for tag in sentence_tags} for sentence_tags in y_pred_llama]

print("Entity-level Micro F1      : {:.4f}".format(seqeval_f1(y_true_llama, y_pred_llama_eval, average='micro')))
print("Entity-level Macro F1      : {:.4f}".format(seqeval_f1(y_true_llama, y_pred_llama_eval, average='macro')))

```

=== EXACT SYSTEM PROMPT ===

You are an expert NLP system performing Named Entity Recognition (NER).

Your task is to classify each word in the provided input array into one of the following CoNLL-2003 categories using the BIO (Begin-Inside-Outside) format.

Entity Categories:

- PER (Person): Names of people (e.g., "John", "Smith").
- ORG (Organization): Companies, agencies, institutions, teams (e.g., "UN", "Google", "Panathinaikos").
- LOC (Location): Countries, cities, states, geographical areas (e.g., "New York", "Greece").
- MISC (Miscellaneous): Nationalities, religions, political parties, events, etc. (e.g., "Spanish", "Olympics").

BIO Format Rules:

- Use 'B-CATEGORY' for the FIRST word of a named entity (e.g., "B-PER").
- Use 'I-CATEGORY' for subsequent words of the SAME entity (e.g., "I-PER").
- Use 'O' for words that are not part of any named entity (e.g., punctuation, verbs).

CRITICAL INSTRUCTIONS:

1. You will receive an array of words.
2. You MUST return a JSON object with exactly one key: "tags". The value must be an array of strings (the BIO tags).
3. The length of the "tags" array MUST EXACTLY match the number of input words. Do not skip any words or punctuation marks!
4. Output ONLY the raw JSON object. Do not include markdown blocks or any conversational text.

=====stdout

Running zero-shot Llama-3.1-8B on 200 sentences...

(Make sure you have a valid GROQ API KEY to run this cell without errors)

```

Groq API calls:   0%|          | 0/200 [00:00<?, ?it/s]
Groq API calls: 100%|██████████| 200/200 [16:43<00:00, 5.02s/it]
=== Llama-3.1-8B Zero-Shot Evaluation (First 200 Sentences) ===
Token-level Micro Accuracy : 0.8614
Entity-level Micro F1      : 0.5189
Entity-level Macro F1      : 0.4047

```

```

/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/seqeval/metrics/sequence_labeling.py:171: UserWarning: ORG seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))

```

In []: %pip install openai

```

In [26]: # =====
# TASK 9B: Zero-Shot gemma4
# =====

import os
import json
import time
from openai import OpenAI
from tqdm.auto import tqdm
from sklearn.metrics import accuracy_score
from seqeval.metrics import f1_score as seqeval_f1

# τρέχει LM Studio σε σταθερό υπολογιστή, η περιέργη IP είναι λόγω tailscale
client = OpenAI(base_url="http://100.124.30.3:1234/v1", api_key="lm-studio")

SYSTEM_PROMPT = """You are an expert NLP system performing Named Entity Recognition (NER).
Your task is to classify each word in the provided input array into one of the following CoNLL-2003 categories using

Entity Categories:
- PER (Person): Names of people (e.g., "John", "Smith").
- ORG (Organization): Companies, agencies, institutions, teams (e.g., "UN", "Google", "Panathinaikos").
- LOC (Location): Countries, cities, states, geographical areas (e.g., "New York", "Greece").
- MISC (Miscellaneous): Nationalities, religions, political parties, events, etc. (e.g., "Spanish", "Olympics").

BIO Format Rules:

```

- Use 'B-CATEGORY' for the FIRST word of a named entity (e.g., "B-PER").
- Use 'I-CATEGORY' for subsequent words of the SAME entity (e.g., "I-PER").
- Use 'O' for words that are not part of any named entity (e.g., punctuation, verbs).

CRITICAL INSTRUCTIONS:

1. You will receive an array of words.
2. You MUST return a JSON object with exactly one key: "tags". The value must be an array of strings (the BIO tags).
3. The length of the "tags" array MUST EXACTLY match the number of input words. Do not skip any words or punctuation.
4. Output ONLY the raw JSON object. Do not include markdown blocks or any conversational text.

```
print("=== EXACT SYSTEM PROMPT ===")
print(SYSTEM_PROMPT)
print("=====\n")

num_sentences = 200
y_true_gemma = []
y_pred_gemma = []

print(f"Running zero-shot gemma4 - e4b (LM Studio) on {num_sentences} sentences...")

for i, sentence_data in enumerate(tqdm(test_sentences[:num_sentences], desc="LM Studio API calls")):
    tokens = sentence_data['tokens']
    true_tags = sentence_data['ner_tags']

    user_prompt = f"Words: {json.dumps(tokens)}"
    error_msg = None

    try:
        completion = client.chat.completions.create(
            # To LM Studio εξυπηρετεί όποιο μοντέλο έχει φορτωθεί στη μνήμη του,
            # ανεξαρτήτως του τι περνάμε εδώ, αλλά κρατάμε το όνομα για λόγους συμβατότητας.
            model="google/gemma-4-e4b",
            messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_prompt}
            ],
            temperature=0.0,
            max_tokens=4096,
            response_format={
                "type": "json_schema",
                "json_schema": {
                    "name": "ner_response",
                    "schema": {
                        "type": "object",
                        "properties": {
                            "tags": {
                                "type": "array",
                                "items": {"type": "string"}
                            }
                        }
                    },
                    "required": ["tags"]
                }
            }
        )

        response_text = completion.choices[0].message.content
        response_json = json.loads(response_text)
        pred_tags = response_json.get("tags", [])
    except Exception as e:
        pred_tags = ["ERROR"] * len(tokens)
        error_msg = str(e)

    # Fallback handling (padding/truncation) to match lengths exactly for sequeval
    if len(pred_tags) < len(tokens):
        pred_tags.extend(['O'] * (len(tokens) - len(pred_tags)))
    elif len(pred_tags) > len(tokens):
        pred_tags = pred_tags[:len(tokens)]

    y_true_gemma.append(true_tags)
    y_pred_gemma.append(pred_tags)

    with open('gemma4_e2b_responses.jsonl', 'a') as f:
        output_data = {'tokens': tokens, 'pred_tags': pred_tags}
        if error_msg:
            output_data['error'] = error_msg
        f.write(json.dumps(output_data) + '\n')

    # Μειωμένο sleep time γιατί δεν υπάρχουν rate limits στο τοπικό inference
    time.sleep(0.1)

print('\n=== gemma4 - e2b Zero-Shot Evaluation (First 200 Sentences) ===')
# Flatten for sklearn metrics
y_true_flat = [tag for sentence_tags in y_true_gemma for tag in sentence_tags]
y_pred_flat = ["O" if tag == "ERROR" else tag for sentence_tags in y_pred_gemma for tag in sentence_tags]
```

```
if len(y_true_flat) > 0:
    print("Token-level Micro Accuracy : {:.4f}".format(accuracy_score(y_true_flat, y_pred_flat)))

# Segeval might throw errors if we have "ERROR" tags
y_pred_gemma_eval = [["0" if (not tag or tag == "ERROR") else str(tag) for tag in sentence_tags] for sentence_tags in sentence_tags_list]

print("Entity-level Micro F1      : {:.4f}".format(segeval_f1(y_true_gemma, y_pred_gemma_eval, average='micro')))
print("Entity-level Macro F1     : {:.4f}".format(segeval_f1(y_true_gemma, y_pred_gemma_eval, average='macro')))
```

=== EXACT SYSTEM PROMPT ===
You are an expert NLP system performing Named Entity Recognition (NER).
Your task is to classify each word in the provided input array into one of the following CoNLL-2003 categories using the BIO (Begin-Inside-Outside) format.

- Entity Categories:
- PER (Person): Names of people (e.g., "John", "Smith").
 - ORG (Organization): Companies, agencies, institutions, teams (e.g., "UN", "Google", "Panathinaikos").
 - LOC (Location): Countries, cities, states, geographical areas (e.g., "New York", "Greece").
 - MISC (Miscellaneous): Nationalities, religions, political parties, events, etc. (e.g., "Spanish", "Olympics").
- BIO Format Rules:
- Use 'B-CATEGORY' for the FIRST word of a named entity (e.g., "B-PER").
 - Use 'I-CATEGORY' for subsequent words of the SAME entity (e.g., "I-PER").
 - Use 'O' for words that are not part of any named entity (e.g., punctuation, verbs).

- CRITICAL INSTRUCTIONS:
1. You will receive an array of words.
 2. You MUST return a JSON object with exactly one key: "tags". The value must be an array of strings (the BIO tags).
 3. The length of the "tags" array MUST EXACTLY match the number of input words. Do not skip any words or punctuation marks!
 4. Output ONLY the raw JSON object. Do not include markdown blocks or any conversational text.

=====

Running zero-shot gemma4 - e4b (LM Studio) on 200 sentences...

```
LM Studio API calls:  0%|          | 0/200 [00:00<?, ?it/s]
LM Studio API calls: 100%|██████████| 200/200 [03:16<00:00, 1.02it/s]
```

=== gemma4 - e2b Zero-Shot Evaluation (First 200 Sentences) ===

```
Token-level Micro Accuracy : 0.8346
Entity-level Micro F1      : 0.4299
Entity-level Macro F1     : 0.2151
```

```
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/segeval/metrics/sequence_labeling.py:171: UserWarning: . seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/segeval/metrics/sequence_labeling.py:171: UserWarning: , seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/segeval/metrics/sequence_labeling.py:171: UserWarning: LOC seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/segeval/metrics/sequence_labeling.py:171: UserWarning: MISC seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/segeval/metrics/sequence_labeling.py:171: UserWarning: ORG seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
```

Task 10: Zero-Shot NER with Llama-3.3-70B

Ανάλυση Κώδικα: Στο παρακάτω κελί κώδικα, επαναλάβαμε ακριβώς τη διαδικασία του Task 9, πραγματοποιώντας δύο μόνο αλλαγές:

1. **Αλλαγή Μοντέλου:** Αντικαταστήσαμε την παράμετρο `model="llama-3.1-8b-instant"` με `model="llama-3.3-70b-versatile"` στο API request της Groq, ώστε να χρησιμοποιήσουμε το κατά πολύ μεγαλύτερο και ικανότερο μοντέλο των 70 δισεκατομμυρίων παραμέτρων.
2. **Μέτρηση Χρόνου (Time Tracking):** Προσθέσαμε μεταβλητές `start_time` και `end_time` (μέσω της βιβλιοθήκης `time`) γύρω από τον βρόχο των 200 API calls, ώστε να καταγράψουμε τον συνολικό χρόνο εκτέλεσης του inference και να συγκρίνουμε τις ταχύτητες.

In [32]:

```
# =====
# TASK 10: Zero-Shot Llama-3.3-70B via Groq
# =====
import time

y_true_llama70 = []
y_pred_llama70 = []

print(f"Running zero-shot Llama-3.3-70B on {num_sentences} sentences...")

SYSTEM_PROMPT = """You are an expert NLP system performing Named Entity Recognition (NER).
Your task is to classify each word in the provided input array into one of the following CoNLL-2003 categories using the BIOES-style format.

Entity Categories:
```

- PER (Person): Names of people (e.g., "John", "Smith").
- ORG (Organization): Companies, agencies, institutions, teams (e.g., "UN", "Google", "Panathinaikos").
- LOC (Location): Countries, cities, states, geographical areas (e.g., "New York", "Greece").
- MISC (Miscellaneous): Nationalities, religions, political parties, events, etc. (e.g., "Spanish", "Olympics").

BIO Format Rules:

- Use 'B-CATEGORY' for the FIRST word of a named entity (e.g., "B-PER").
- Use 'I-CATEGORY' for subsequent words of the SAME entity (e.g., "I-PER").
- Use 'O' for words that are not part of any named entity (e.g., punctuation, verbs).

CRITICAL INSTRUCTIONS:

1. You will receive an array of words.
2. You MUST return a JSON object with exactly one key: "tags". The value must be an array of strings (the BIO tags).
3. The length of the "tags" array MUST EXACTLY match the number of input words. Do not skip any words or punctuation.
4. Output ONLY the raw JSON object. Do not include markdown blocks or any conversational text.

"""

```
start_time = time.time()
```

```
for i, sentence_data in enumerate(tqdm(test_sentences[:num_sentences], desc="Groq 70B API calls")):
```

```
    tokens = sentence_data['tokens']
```

```
    true_tags = sentence_data['ner_tags']
```

```
    user_prompt = f"Words: {json.dumps(tokens)}"
```

```
    error_msg = None
```

```
    try:
```

```
        completion = client.chat.completions.create(
```

```
            model="llama-3.3-70b-versatile",
```

```
            messages=[
```

```
                {"role": "system", "content": SYSTEM_PROMPT},
```

```
                {"role": "user", "content": user_prompt}
```

```
            ],
```

```
            temperature=0.0,
```

```
            max_tokens=4096,
```

```
            response_format={
```

```
                "type": "json_schema",
```

```
                "json_schema": {
```

```
                    "name": "ner_response",
```

```
                    "schema": {
```

```
                        "type": "object",
```

```
                        "properties": {
```

```
                            "tags": {
```

```
                                "type": "array",
```

```
                                "items": {"type": "string"}
```

```
                            }
```

```
                        },
```

```
                        "required": ["tags"]
```

```
                    }
```

```
                }
```

```
            }
```

```
        )
```

```
        response_text = completion.choices[0].message.content
```

```
        response_json = json.loads(response_text)
```

```
        pred_tags = response_json.get("tags", [])
```

```
    except Exception as e:
```

```
        pred_tags = ["ERROR"] * len(tokens)
```

```
        error_msg = str(e)
```

```
    # Fallback handling
```

```
    if len(pred_tags) < len(tokens):
```

```
        pred_tags.extend(['O'] * (len(tokens) - len(pred_tags)))
```

```
    elif len(pred_tags) > len(tokens):
```

```
        pred_tags = pred_tags[:len(tokens)]
```

```
    y_true_llama70.append(true_tags)
```

```
    y_pred_llama70.append(pred_tags)
```

```
    with open('llama_70b_responses.jsonl', 'a') as f:
```

```
        output_data = {'tokens': tokens, 'pred_tags': pred_tags}
```

```
        if error_msg:
```

```
            output_data['error'] = error_msg
```

```
        f.write(json.dumps(output_data) + '\n')
```

```
    time.sleep(2.5)
```

```
end_time = time.time()
```

```
total_time = end_time - start_time
```

```
print('\n=== Llama-3.3-70B Zero-Shot Evaluation ===')
```

```
print(f"Total Execution Time (200 sentences): {total_time:.2f} seconds")
```

```
y_true_flat_70 = [tag for sentence_tags in y_true_llama70 for tag in sentence_tags]
```

```
y_pred_flat_70 = ["O" if tag == "ERROR" else tag for sentence_tags in y_pred_llama70 for tag in sentence_tags]
```

```
if len(y_true_flat_70) > 0:
```



```
print("Token-level Micro Accuracy : {:.4f}".format(accuracy_score(y_true_flat_70, y_pred_flat_70)))

y_pred_llama70_eval = [{"0" if tag == "ERROR" else tag for tag in sentence_tags} for sentence_tags in y_pred_llama70

print("Entity-level Micro F1      : {:.4f}".format(seqeval_f1(y_true_llama70, y_pred_llama70_eval, average='micro')))
print("Entity-level Macro F1     : {:.4f}".format(seqeval_f1(y_true_llama70, y_pred_llama70_eval, average='macro')))
```

Running zero-shot Llama-3.3-70B on 200 sentences...

Groq 70B API calls: 100%|██████████| 200/200 [11:22<00:00, 3.41s/it]

=== Llama-3.3-70B Zero-Shot Evaluation ===

Total Execution Time (200 sentences): 682.97 seconds

Token-level Micro Accuracy : 0.8346

Entity-level Micro F1 : 0.4299

Entity-level Macro F1 : 0.2151

```
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/seqeval/metrics/sequence_labeling.py:171: UserWarning: . seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/seqeval/metrics/sequence_labeling.py:171: UserWarning: , seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/seqeval/metrics/sequence_labeling.py:171: UserWarning: LOC seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/seqeval/metrics/sequence_labeling.py:171: UserWarning: MISC seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
/home/george/MScAI/NLP-3/NLP-3/.venv/lib64/python3.14/site-packages/seqeval/metrics/sequence_labeling.py:171: UserWarning: ORG seems not to be NE tag.
  warnings.warn('{} seems not to be NE tag.'.format(chunk))
```

1o prompt - minimal

NER. Categories: PER, ORG, LOC, MISC. Return JSON object with key "tags" mapping to array of BIO tags. Array length MUST exactly match words length.

=== Llama-3.1-8B Zero-Shot Evaluation (First 200 Sentences) === Token-level Micro Accuracy : 0.2656 Entity-level Micro F1 : 0.0048 Entity-level Macro F1 : 0.0054

=== Llama-3.3-70B Zero-Shot Evaluation === Total Execution Time (200 sentences): 606.54 seconds Token-level Micro Accuracy : 0.8233 Entity-level Micro F1 : 0.5781 Entity-level Macro F1 : 0.2809

=== gemma4 - e2b Zero-Shot Evaluation (First 200 Sentences) === Token-level Micro Accuracy : 0.6276 Entity-level Micro F1 : 0.0937 Entity-level Macro F1 : 0.0618

2o prompt - αναλυτικό

You are an expert NLP system performing Named Entity Recognition (NER). Your task is to classify each word in the provided input array into one of the following CoNLL-2003 categories using the BIO (Begin-Inside-Outside) format.

Entity Categories:

- PER (Person): Names of people (e.g., "John", "Smith").
- ORG (Organization): Companies, agencies, institutions, teams (e.g., "UN", "Google", "Panathinaikos").
- LOC (Location): Countries, cities, states, geographical areas (e.g., "New York", "Greece").
- MISC (Miscellaneous): Nationalities, religions, political parties, events, etc. (e.g., "Spanish", "Olympics").

BIO Format Rules:

- Use 'B-CATEGORY' for the FIRST word of a named entity (e.g., "B-PER").
- Use 'I-CATEGORY' for subsequent words of the SAME entity (e.g., "I-PER").
- Use 'O' for words that are not part of any named entity (e.g., punctuation, verbs).

CRITICAL INSTRUCTIONS:

1. You will receive an array of words.
2. You MUST return a JSON object with exactly one key: "tags". The value must be an array of strings (the BIO tags).
3. The length of the "tags" array MUST EXACTLY match the number of input words. Do not skip any words or punctuation marks!
4. Output ONLY the raw JSON object. Do not include markdown blocks or any conversational text.

=== Llama-3.1-8B Zero-Shot Evaluation (First 200 Sentences) === Token-level Micro Accuracy : 0.8614 Entity-level Micro F1 : 0.5189 Entity-level Macro F1 : 0.4047

=== gemma4 - e2b Zero-Shot Evaluation (First 200 Sentences) === Token-level Micro Accuracy : 0.8346 Entity-level Micro F1 : 0.4299 Entity-level Macro F1 : 0.2151

=== Llama-3.3-70B Zero-Shot Evaluation === Total Execution Time (200 sentences): 682.97 seconds Token-level Micro Accuracy : 0.8346 Entity-level Micro F1 : 0.4299 Entity-level Macro F1 : 0.2151

Συγκριτική Ανάλυση 4 Μοντέλων: Απόδοση vs Ταχύτητα

Έχοντας πλέον αξιολογήσει 4 διαφορετικά μοντέλα στο Named Entity Recognition (NER), εξάγουμε τα εξής συμπεράσματα συγκρίνοντας την απόδοση (F1-score) και τον χρόνο (Speed):

Μοντέλο	Προσέγγιση	Απόδοση (Metrics)	Ταχύτητα & Πόροι (Speed)
BERT-base	Fine-Tuning	Υψηλή (συνήθως F1 ~90-91%). Το μοντέλο προσαρμόστηκε στα δεδομένα, μαθαίνοντας άριστα τις BIO δομές.	Μέτρια/Γρήγορη στο inference. Απαιτεί όμως σημαντικό χρόνο εκπαίδευσης (Fine-Tuning) πριν τη χρήση.
RoBERTa-base	Fine-Tuning	Πολύ Υψηλή (συχνά ξεπερνά το BERT κατά 1-2%). Η έλλειψη NSP και ο τεράστιος όγκος pre-training του δίνουν προβάδισμα.	Παρόμοια γρήγορη ταχύτητα inference με το BERT. Απαιτεί επίσης χρόνο για το Fine-Tuning.
Llama-3.1 (8B)	Zero-Shot Prompting	Χαμηλή. Αντιμετωπίζει δυσκολίες στο αυστηρό alignment του BIO format και στους ιδιωτισμούς του CoNLL003.	Χαμηλή ταχύτητα (υψηλό latency) λόγω σειριακών API calls και autoregressive παραγωγής. Κανένας χρόνος εκπαίδευσης (άμεση χρήση).
Llama-3.3 (70B)	Zero-Shot Prompting	Καλύτερη από το 8B (βελτιωμένη σημασιολογική κατανόηση), αλλά εξακολουθεί να μην φτάνει τα fine-tuned BERT/RoBERTa στα αυστηρά sequeval metrics.	Ακόμα χαμηλότερη ταχύτητα και υψηλότερο κόστος (API tokens), καθώς πρόκειται για μοντέλο ~10x μεγαλύτερο.

Συμπέρασμα: Τα Fine-Tuned μοντέλα πετυχαίνουν σαφώς καλύτερη ισορροπία Απόδοσης / Ταχύτητας Inference στο συγκεκριμένο task.

Θεωρητικό Συμπέρασμα: Decoder-only vs Encoder-only στο NER

Πόσο ανταγωνιστικά είναι τα decoder-only μοντέλα (όπως το Llama) απέναντι στα encoder-only (όπως τα BERT/RoBERTa) για tasks ταξινόμησης ανά λέξη (Token Classification / NER);

1. Κατανόηση Περιβάλλοντος (Context Understanding):\

- Τα **Encoder-only** μοντέλα (BERT) χρησιμοποιούν bidirectional attention, πράγμα που σημαίνει ότι, όταν "διαβάζουν" μια λέξη (π.χ. στη μέση της πρότασης), λαμβάνουν ταυτόχρονα υπόψη και το αριστερό αλλά και το δεξί συγκείμενο (τι προηγείται και τι έπεται). Αυτό είναι κρίσιμο για το NER, καθώς η σημασία μιας λέξης εξαρτάται άμεσα από τη συνέχεια της πρότασης.
- Τα **Decoder-only** μοντέλα (Llama, Gemma) χρησιμοποιούν causal (masked) attention, με προσανατολισμό την πρόβλεψη της επόμενης λέξης. Για να κατανοήσουν το δεξί συγκείμενο σε zero-shot μορφή, πρέπει πρακτικά να επεξεργαστούν όλο το prompt εξ αρχής και να δημιουργήσουν την απάντηση μεταγενέστερα (autoregressively). Αν και τα σύγχρονα LLMs το καταφέρνουν εξαιρετικά, αρχιτεκτονικά είναι μια λιγότερο φυσική/αποδοτική μέθοδος για αμιγώς token-level εργασίες.

2. Zero-Shot ικανότητες vs Ανάγκη Fine-Tuning:\

- Το ασύγκριτο πλεονέκτημα των LLMs (Decoder-only) είναι η ικανότητά τους να αποδίδουν άμεσα, χωρίς κανένα fine-tuning (Zero-Shot). Μπορείς απλώς να περιγράψεις το task στο prompt, και αυτά θα ανταποκριθούν.
- Στον αντίποδα, το BERT είναι πρακτικά άχρηστο σε μορφή zero-shot για NER. Απαιτεί αυστηρά ένα labeled dataset, εκπαίδευση (fine-tuning) μιας νέας ταξινομικής κεφαλής (classification head) και υπολογιστικούς πόρους (GPU) για την προετοιμασία του.

3. Computational Cost & Time Cost (Κόστος & Ταχύτητα):\

- Για παραγωγή μεγάλης κλίμακας (production inference), η κλήση ενός LLM API (έστω και του 8B) για κάθε μεμονωμένη πρόταση ή έγγραφο είναι υπερβολικά ακριβή και αργή (high latency).
- Ένα fine-tuned BERT/RoBERTa έχει μόλις ~110M παραμέτρους. Εκτελεί inference σε ελάχιστα χιλιοστά του δευτερολέπτου, μπορεί να φιλοξενηθεί εύκολα και δωρεάν σε τοπικούς servers ή ακόμα και κινητά τηλέφωνα, εξάγοντας ακαριαία predictions για τεράστια έγγραφα.

4. Πρόσφατη πρόοδος στα decoder-only μοντέλα:

Επιπλέον των ζητούμενων, δοκίμασα και το Gemma 4-e4b, το οποίο είναι επίσης μοντέλο 8B παραμέτρων. Έτρεξε αποκλειστικά σε local περιβάλλον (σε σταθερό υπολογιστή σε LMStudio πάνω σε rtx4060 8Gb), απαιτώντας σημαντικά λιγότερους πόρους από πλευράς μνήμης (5.4GB σε σύγκριση με τα 9.6GB που ζητά το μικρό Llama) και πετυχαίνοντας δραματικά καλύτερα αποτελέσματα από το μικρό Llama με το minimal prompt.

Φτάνοντας στην δοκιμή του αναληπτικού prompt είχα ήδη βελτιστοποιήσει τον κώδικα στο μέγιστο των δυνατοτήτων μου ώστε να μαζεύω τα λάθη του μικρού Llama, οπότε τα αποτελέσματα έχουν βαρύ bias υπέρ του.

Παρόλο που εν τέλει υστερεί και το gemma4 σε σχέση με τα encoder only μοντέλα για τέτοιου είδους tasks, είναι εμφανές ότι τα decoder μοντέλα έχουν κάνει αλματώδη πρόοδο την τελευταία διετία και το χάσμα της απόδοσης τους κλείνει.

Τελικό Συμπέρασμα:

Για **εξειδικευμένα, αυστηρά** token classification tasks όπως το CoNLL003 NER, τα μικρά **Encoder-only μοντέλα εξακολουθούν να κυριαρχούν** (state-of-the-art απόδοση με ελάχιστο κόστος inference), εφόσον διαθέτουμε επαρκή δεδομένα εκπαίδευσης.

Αντίθετα, τα μεγάλα **Decoder-only μοντέλα** είναι εξαιρετικά πολύτιμα όταν δεν διαθέτουμε δεδομένα (Zero-Shot / Few-Shot scenarios), όταν η αναγνώριση οντοτήτων απαιτεί ακραία λογική, ή όταν τα χρησιμοποιούμε offline για να δημιουργήσουν αυτόματα συνθετικά δεδομένα (synthetic labels), με τα οποία στη συνέχεια θα εκπαιδύσουμε έναν γρήγορο encoder!

Πειραματικά Αποτελέσματα που μου φάνηκαν απροσδόκητα

Μου έκανε μεγάλη εντύπωση η απόδοση του frozen-weight μοντέλου όταν χρησιμοποίησα το αρχικό learning rate, $1e-5$. Η πτώση ήταν θεαματική, με νούμερα παρόμοια του llama 3.1 8B με το minimal prompt, το οποίο στο 1/3 των προτάσεων ξέφευγε από τις οδηγίες! Το γεγονός αυτό μου δείχνει πόση σημασία έχει η σωστή ρύθμιση των υπερπαραμέτρων.